

AI大模型原理和应用面试题速记通关版 _ 面试刷题

mianshiya.com

本资源来自面试鸭：<https://www.mianshiya.com>

推荐更多免费学编程资源：

1. 编程导航学习网站：[学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库：[开始刷题，面试遇原题!](#)
3. 精选简历模板大全：[1分钟搞定简历!](#)
4. AI 资源导航网站：[获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试：[随时随地提升面试能力](#)

请解释大模型微调(Fine-tuning)的原理，并说明在什么业务场景下需要微调而不是直接使用基础模型？

微调的本质是**参数高效迁移**，不是从头训练。大模型已经在海量通用语料上学到了语言规律和世界知识，微调是在这个基础上，用特定领域的数据去调整模型参数，让它适应新任务。

- 1) 你拿到一个预训练好的大模型，比如 LLaMA 或 Qwen，它的“常识”已经很完整。但如果你要做金融研报生成，它可能写得不够专业。这时候直接 prompt 工程搞不定，因为领域术语、行文逻辑差异太大。
- 2) 微调就是拿一批金融报告数据，用有监督的方式继续训练。比如输入“某公司营收增长30%”，让模型输出对应的研报段落。反向传播会更新权重，让模型在该任务上表现更好。
- 3) 关键在于，并不是所有层都要重训。现在很多做法是只微调部分参数，比如 LoRA (Low-Rank Adaptation)，冻结原模型权重，只训练低秩矩阵。这样显存占用少，训练快，适合业务迭代。

需要微调的场景通常是：

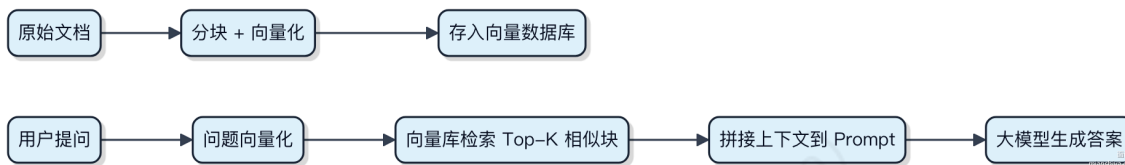
- 输出格式/风格强约束，比如法律文书、医疗诊断记录
- 领域知识密集，基础模型没见过的专业术语体系
- 企业私有数据不能走 API，必须本地部署并定制

如果只是简单问答、摘要生成，且对表达风格要求不高，那其实用提示词+RAG就够了。微调成本高，周期长，真要上得想清楚是不是非用不可。

RAG 的完整流程是怎么样的？

RAG 的流程其实就三步：准备数据、查资料、生成答案。整个过程让大模型在回答前，先去“翻书”找依据。

- 1) 数据准备阶段，先把外部知识库的文档切片，比如把 PDF 或数据库表结构说明拆成一段段文本块。接着用 embedding 模型给每一块转成向量，存进向量数据库，像 Milvus 或 Pinecone 这类工具干的就是这事。
- 2) 用户提问时，系统不会直接扔给大模型。而是先把问题也转成向量，去向量库里做相似度搜索，找出最相关的几个文本块。这一步很关键，决定了模型能“看到”哪些背景知识。
- 3) 最后，把这些检索到的文本块拼接成提示词，连同原问题一起喂给大模型。模型基于这些上下文生成答案，而不是靠记忆瞎编。这样出来的回答有据可查，幻觉少。



这个流程里，embedding 模型和向量检索的质量直接决定效果。如果检索错了，后面再强的模型也搞不定。常见坑是文本分块不合理，比如按固定字符切，把一个完整概念切碎了，导致查不准。

什么自查询？为什么在 RAG 中需要自查询？

自查询其实是让模型自己生成查询条件，去检索它需要的信息。比如你问“苹果最新财报怎么样”，模型不会直接回答，而是先拆解问题，生成像“苹果 2024 年 Q2 营收、利润、同比增长”这样的检索关键词，再拿这些词去查文档。

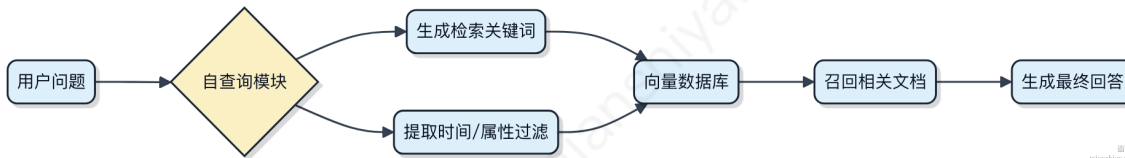
这在 RAG 里特别关键，因为原始问题往往太笼统或包含多个维度。直接拿原问题去搜，召回的内容可能不精准。自查询相当于加了一层智能路由，把复杂问题翻译成适合检索的结构化查询语句。

- 1) 能处理时间范围过滤，比如“去年销售额最高的产品”会被转成带时间约束的查询
- 2) 支持属性筛选，“便宜的安卓手机”会拆成“价格低于 2000 元、操作系统为 Android”
- 3) 适配向量库的元数据检索，比如用 Chroma 或 Pinecone 的 where 条件过滤

代码上一般用 LLM 接一个提示模板：

```
String prompt = "根据用户问题生成检索查询语句: " + userQuestion;
List<String> queries = llm.generate(prompt);
List<Document> results = vectorStore.search(queries);
```

整个流程像是给 RAG 装了个“理解大脑”，先把人话转化成机器能高效检索的指令，避免压根不经过语义解析就硬搜。



什么提示压缩？为什么在 RAG 中需要提示压缩？

提示压缩其实就是把 RAG 中过长的上下文输入给“瘦身”。你想想，大模型的上下文长度是有限的，比如 32K 或 128K，但检索回来的文档片段一多，加上原始问题和模板，很容易就超了。

- 1) 最直接的问题就是 token 数爆掉，请求直接被拒，压根不经过模型推理。
- 2) 就算没超，长上下文推理成本高，响应慢，延迟上去用户体验就差了。
- 3) 而且很多检索出的内容其实跟问题关系不大，纯属噪音，反而干扰生成质量。

所以得做提示压缩，把那些**冗余信息**、重复段落、无关句子给剔除或简化。不是简单删减，而是保留关键事实和语义，让输入更紧凑高效。

常见的做法有几种：

- 用小模型先做一轮重排序或摘要，比如用 BGE 或 Cohere reranker 挑最相关的 top-k。
- 在 prompt 组装时加逻辑，只保留高分 chunk，甚至对每个 chunk 做句子级裁剪。
- 也可以引入类似 **HyDE** 的思路，先生成假设答案再反向匹配，过滤低相关度内容。

代码上没啥复杂语法，核心是控制组装后的总 token 数：

```
# 伪代码示意
context = "\n".join([chunk["text"] for chunk in relevant_chunks[:3]]) # 限制数量
prompt = f"基于以下内容回答：\n{context}\n\n问题：{query}"
```

要不要压缩，其实看场景。如果检索精度高、返回少，可能不需要。但面对海量文档库，比如用在企业知识库搜寻，就得靠这招扛住长上下文压力。

什么是 RAG 中的分块？为什么需要分块？

RAG 的上下文长度是有限的，比如主流模型一般支持 32k 或 128k 的 token 上下文。你塞进去的 prompt 越长，能留给检索内容的空间就越少。分块其实就是把原始文档切成适合模型处理的小段落，让每次检索和生成都能在上下文窗口内完成。

- 1) 分块不是简单按段落切。如果按固定字符数切，可能把一句话从中间劈开，语义就断了。好的分块策略会尽量保持语义完整，比如用 **递归分割**，先按标题切，再按段落，最后才是固定长度。LangChain 里的 `RecursiveCharacterTextSplitter` 就是这么干的。
- 2) 块太小，信息不全，模型容易“只见树木不见森林”；块太大，又挤占上下文，影响其他 prompt 内容。一般来说，512 到 1024 个 token 是比较常见的块大小，具体得看你的数据和模型。
- 3) 有些场景还得考虑跨块关联。比如一个技术方案分布在三个块里，单靠一个块检索出来，模型也拼不全逻辑。这时候就得靠后续的查询扩展或多步检索来补救。

代码上大概长这样：

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=50,
    separators=["\n\n", "\n", ".", " ", ""],
)
docs = splitter.split_text(large_document)
```

要不要分块，其实取决于你的数据形态和检索目标。像 PDF、长网页这种肯定要分，但如果是短问答对，可能直接喂进去就行。

在 RAG 中，常见的分块策略有哪些？分别有什么区别？

分块策略直接影响检索效果，选不好后面全白搭。关键得看文本的语义连贯性和上下文依赖。

1) 固定大小分块

按字符或 token 数硬切，比如每 512 个 token 切一块。简单粗暴，适合结构规整的文档。但容易把一句话从中间劈开，语义被撕裂，检索回来的片段可能压根不经过原意。像 PDF 或网页正文这种连贯性不强的还行，但技术文档、论文就搞不定。

2) 基于段落或标题分割

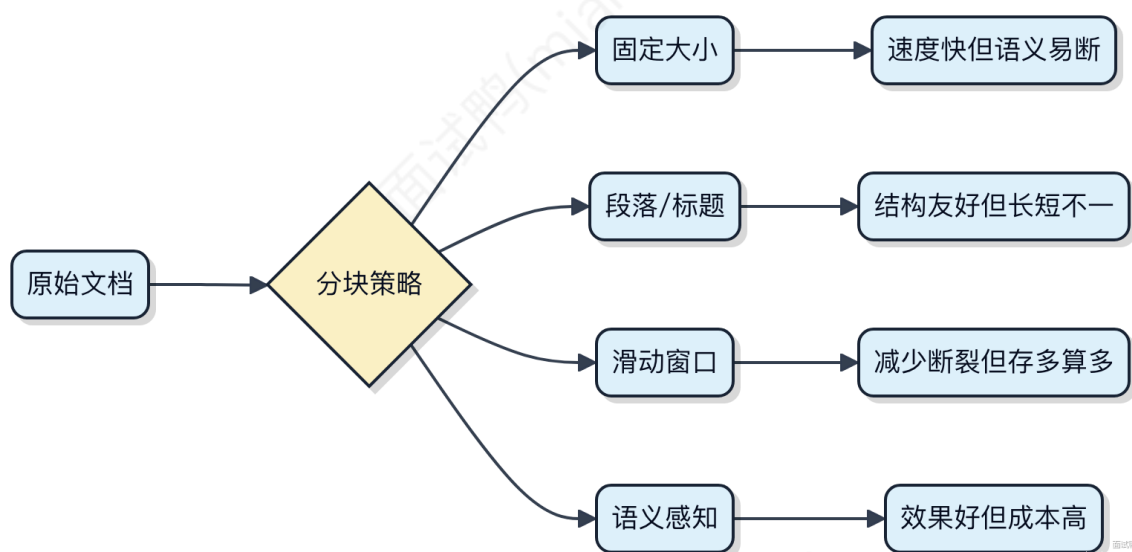
利用自然断点，比如一级/二级标题之间、空行处分割。能保留完整语义单元，适合 Markdown、Word 这类有结构的文档。问题在于块大小不均匀，有的标题下内容贼长，检索时信息密度太低。

3) 滑动窗口重叠分块

在固定大小基础上，让相邻块有部分重叠，比如每次滑动 512，重叠 128。这样能缓解语义断裂，关键信息大概率出现在多个块里。但会增加索引量和检索耗时，重复内容多，存储成本翻倍也不稀奇。

4) 语义感知分块（如 RecursiveCharacterTextSplitter）

先按标点切，再按长度合并，尽量保证句子完整。LangChain 里默认用这个，实际项目中表现最稳。更进一步可以用嵌入模型做句子相似度判断，在语义边界处分割，不过延迟高，一般用在对精度要求极高的场景。



选哪种？短平快任务用滑动窗口+固定大小，追求效果上语义分块，有明确结构就优先标题分割。

在 RAG 中的 Embedding 嵌入是什么？

Embedding 在 RAG 里干的事，就是把文本转成机器能算的“数字向量”。你给它一句话、一段文档，它就输出一个长长的浮点数数组，这个数组得能保留原文的语义信息。

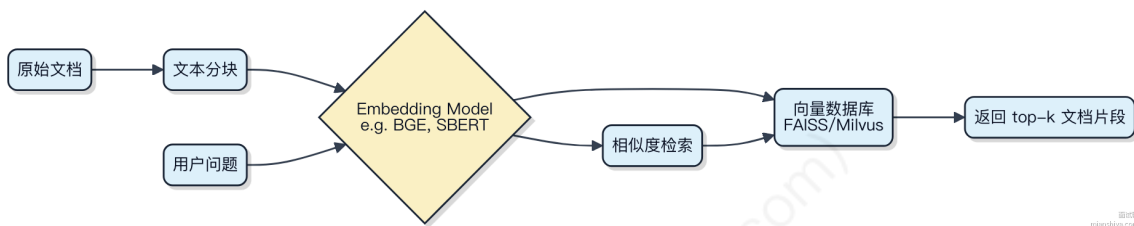
比如你搜“怎么修自行车”，系统其实不是在关键词匹配，而是先把这句话做个 Embedding 变成向量，再去向量空间里找和它最接近的那些文档片段。像用百度搜索那种关键词匹配早就跟不上了，现在都靠这种语义相似度来找答案。

1) 这活一般由预训练模型来干，比如 BERT、Sentence-BERT (SBERT)，或者专门做检索的 bge 模型。它们会把不同长度的文本压到固定维度的向量里，常见的是 768 或 1024 维。

2) RAG 流程中，Embedding 主要用于两个地方：一是离线把知识库文档切片后全部向量化存进向量数据库，比如用 FAISS、Milvus 或 Pinecone；二是线上用户一提问，立刻转成向量去库里查 top-k 最相似的 chunk。

3) 效果好不好，关键看 Embedding 是否真能把相近意思的文本“拍”到向量空间里挨得近。如果“如何重启路由器”和“拔电源再插上”这两个句子向量距离很近，那说明嵌入质量高。

代码上基本就是调个 model.encode(text) 就完事，但背后模型选型、是否微调、向量索引构建方式都会影响最终召回率。



在 RAG 中，你知道有哪些 Embedding Model 嵌入模型？

Embedding 模型在 RAG 里头其实就干一件事：把文本转成向量，让检索系统能算相似度。选对模型直接影响召回质量。

主流的通用嵌入模型有 Sentence-BERT (SBERT)，它在 BERT 基础上加了个池化层，专门优化句子级表示。比如你用 paraphrase-MiniLM-L6-v2，小模型快，适合轻量场景。

进阶一点的是 Cohere 提供的 multilingual-2 和 command-r 系列，尤其是多语言支持好，在跨语言检索里表现稳。Cohere 的 API 易用，但得走网络请求。

还有 OpenAI 的 text-embedding-ada-002，虽然不开放本地部署，但在英文任务上精度高，适合不想调参直接上生产的场景。

开源界现在用得猛的是 **BGE**（来自智源研究院），比如 bge-small-zh-v1.5，中文场景下效果拉满，本地跑起来也顺。它通过“查询-文档”对增强训练，特别适配 RAG 的检索偏好。

别忘了还有 Jina AI 推的 jina-embeddings-v2，支持 8192 token 长度，长文本处理有优势，而且许可友好，能商用。

选模型得看三点：语言是否匹配、延迟能不能压住、有没有长文本需求。别一上来就用大模型，小模型蒸馏过的在 90% 场景都够用。



实际项目里，BGE + Milvus 是中文 RAG 的常见组合，本地可控又高效。

什么是 MCP 协议，它在 AI 大模型系统中的作用是什么？

MCP 协议并不是一个在 AI 大模型领域广泛通用的标准协议，目前也没有权威文献或主流框架将其定义为类似 HTTP、gRPC 那样的基础通信协议。它更可能是指特定厂商或系统内部定义的 **模型控制平面 (Model Control Plane)** 协议，用于协调大模型服务中的调度、分发与生命周期管理。

在典型的推理服务平台比如阿里云百炼、vLLM 或 Triton Inference Server 中，这类“控制协议”主要干三件事：

- 1) 接收推理请求并做路由决策，比如该发给哪个模型实例、是否需要扩缩容
- 2) 管理模型加载、卸载、版本切换，确保 GPU 资源高效利用
- 3) 收集监控指标，配合实现弹性伸缩和故障转移

你可以把它理解成 K8s 里的 kubelet 和 API Server 之间的通信逻辑，只不过面向的是模型实例而不是容器。真正的请求处理——也就是数据平面 (Data Plane)——还是靠 gRPC 或 REST 承载实际的 token 流量，而 MCP 这类控制指令通常走独立通道，频率低但关键性高。

```
// 伪代码示意：控制平面接收加载模型指令
void onReceiveLoadModel(String modelId, String version) {
    ModelInstance instance = createOrReuseInstance(modelId);
    instance.loadFromStorage(); // 可能涉及 mmap、量化加载等优化
    registerToRouter(instance); // 注册到流量路由器
}
```

要不要用这类协议，取决于系统规模。小场景直接 API 调用就够了，上了百个模型实例后，就得靠统一控制平面来“扛住”运维复杂度了。

MCP 架构包含哪些核心组件？

MCP 架构这个词在主流技术领域没有统一标准定义，更常见的类似缩写是 MCP（Model-View-Presenter）属于 UI 架构模式，常用于客户端开发。但如果你指的是服务端或中间件场景下的某种定制架构，可能是特定团队对 **消息驱动、控制平面分离** 类系统的简称。

不过从实际系统设计角度看，如果把 MCP 理解为 **Message-Driven Control Plane** 这类架构，它的核心组件一般包含三个部分：

1) 消息中枢

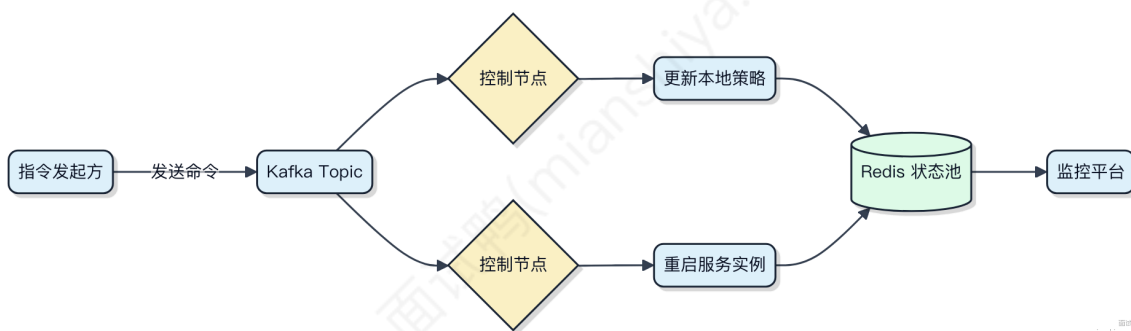
负责路由和分发指令，比如用 Kafka 或 RocketMQ 做命令总线，所有控制指令通过 Topic 广播。消费者按需订阅，实现解耦。这种设计在微服务治理中很常见，像 Nacos 配置变更就是靠长轮询 + 消息通知双机制推到客户端。

2) 控制单元

接收消息后执行具体逻辑，比如修改本地状态、调用外部接口、触发任务调度。它不直接暴露 HTTP 接口，而是被动响应事件。Spring Cloud Stream 的 @StreamListener 就是典型写法，压根不经过 Web 层。

3) 状态反馈器

反向上报执行结果，形成闭环。通常往一个集中式存储写状态，比如把节点健康度打到 Redis，或者把执行日志发到 Elasticsearch。ZooKeeper 临时节点也干这事，会话一断自动清理状态。



MCP 协议支持哪两种模式？

MCP（Model Context Protocol）目前主要支持两种交互模式，对应不同的上下文处理方式。

1) 请求-响应模式

这是最常用的模式，客户端发送一个请求，携带特定上下文，服务端完全基于该上下文生成响应，交互结束。整个过程类似 HTTP 的调用模型，适合一次性任务，比如让大模型总结一段文本、翻译一句话。这种模式下，上下文只在当前请求中生效，不会保留。

2) 会话模式

系统会维护一个持续的上下文会话状态，客户端可以连续发送多条消息，每条都基于之前的对话历史进行理解与回应。典型场景是聊天机器人、AI 助手这类需要记忆上下文的产品，比如使用 MCP 协议对接 LLM 构建客服系统时，用户前几轮提到的信息要能在后续对话中被引用。

会话模式的关键在于服务端需要管理 Session 状态，可能结合 UUID 或 token 来标识不同用户的上下文隔离。而请求-响应模式无状态，更容易水平扩展。

要不要用会话模式，得看业务是否需要“记住”前面的内容。如果只是做离散任务，比如内容审核、关键词提取，压根不需要维持状态，用第一种就够了。

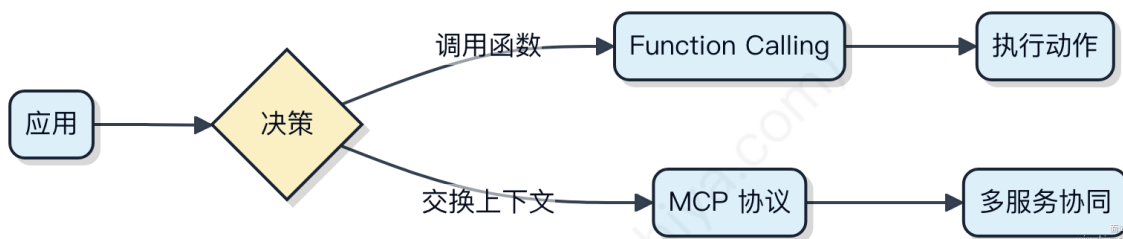
MCP 与 Function Calling 的区别是什么？

MCP 和 Function Calling 看似都在让模型“调用功能”，但它们的定位和机制完全不同。

MCP 其实是一种协议，更准确地说它是 **Model Context Protocol**，它定义了外部系统如何向大模型提供上下文、工具描述以及如何接收模型的结构化输出。它不绑定具体实现，强调的是模型与环境之间的标准化交互方式。你可以把它理解成一个通用的“插件通信规范”，像 GitHub Copilot Extensions 就基于这类思想构建扩展能力。

而 Function Calling 是具体的能力，指的是大模型能根据输入决定是否调用某个预定义函数，并生成符合 schema 的参数。主流闭源模型如 GPT-4 都支持这个特性，它是实现 Agent 行为的基础组件之一。比如你用 LangChain 写个天气查询，模型会判断用户意图后返回 `{"name": "get_weather", "arguments": {"city": "Beijing"}}` 这样的结构。

关键区别在于：Function Calling 是单一模型对外暴露的能力接口，MCP 则是多个系统间协作的协议框架，支持更复杂的双向交互和资源共享。



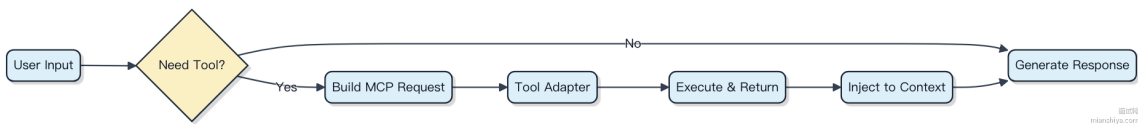
MCP 的工作流程是什么？

MCP 其实是 Model Context Protocol 的缩写，它不是传统中间件或 Java 领域的标准组件，而是在 AI Agent 架构中用来规范大模型与外部工具、数据源之间交互的协议。它的核心作用是让模型能安全、可控地访问运行时上下文。

整个流程从用户发起请求开始。Agent 接收到输入后，会先做一次意图解析，判断是否需要调用工具。如果需要，就会构造一个符合 MCP 格式的请求包，里面包含动作类型、参数和上下文元信息。

接下来这个请求会被转发给对应的工具适配器。比如你要查数据库，就会走 SQL 工具链；要发 HTTP 请求，就交给 API 客户端处理。执行结果原路返回，由 MCP 解析器把原始响应转成模型能理解的自然语言片段。

- 1) 请求触发：用户输入进入 Agent 循环
- 2) 决策判断：Controller 判断是否需外部调用
- 3) 协议封装：按 MCP 标准生成 tool call 消息
- 4) 工具执行：Adapter 调用具体服务并获取结果
- 5) 上下文注入：将结果以结构化方式回填到 prompt 上下文



这种设计的好处是解耦了模型逻辑和外部系统。像 LangChain 或 Semantic Kernel 这类框架，底层其实就是靠类似 MCP 的机制在做调度。你不需要改模型代码，只要注册新工具就能扩展能力。

在 Spring AI 框架中如何集成 MCP?

Spring AI 框架本身是 Spring 生态中用于接入大模型服务的抽象层，它的设计目标是让 Java 应用能统一方式调用不同 AI 平台的能力。MCP (Model Context Protocol) 是一种用于在客户端与模型服务之间传递上下文信息的协议，常见于某些私有化部署或定制化推理平台。

要把 MCP 集成进 Spring AI，关键在于**自定义客户端适配器**。Spring AI 支持扩展 `ChatClient` 接口，你可以基于 `RestClient` 或 `WebClient` 实现一个专用于支持 MCP 协议的服务通信的客户端。

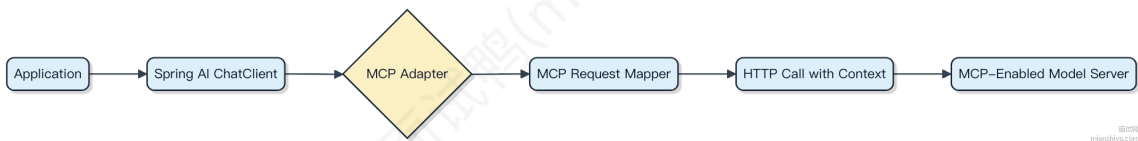
1) 你需要先定义请求结构体，匹配 MCP 规范里的字段，比如 `context`，`prompt`，`model` 等。2) 通过 `@RegisterAiClient` 注解注册你的实现，或者手动配置为 `Bean`。3) 在调用时注入 `ChatClient`，框架会自动路由到你的 MCP 适配器。

```

@Bean
ChatClient mcpChatClient(WebClient webClient) {
    return ChatClient.builder()
        .baseUrl("https://mcp-gateway.example.com")
        .webClient(webClient)
        .build();
}
  
```

实际请求中，header 或 payload 里要带上 MCP 要求的元数据，比如会话 ID、租户上下文等。这类脏活累活一般封装在拦截器里处理。

如果使用 Spring AI 0.8.0+ 版本，已经支持 SPI 扩展机制，可以更干净地插拔协议实现。不过目前主流厂商如通义千问、百川都不走 MCP，它更多出现在内部统一推理网关场景，比如阿里云灵积的部分私有实例。



MCP 协议安全性设计包含哪些层面?

MCP 协议本身并不是一个广泛标准化的通用协议，更多是特定系统或平台内部定义的通信规范。讨论其安全性设计，得从典型中间件通信协议的防护思路入手，一般会覆盖这几个层面。

传输安全靠 TLS/SSL 加密链路，防止数据被窃听或篡改，这是基础操作。身份认证通常走双向证书、API Key 或 OAuth 令牌，确保连接双方都是合法角色。访问控制会在协议层嵌入权限信息，比如 Kafka 的 ACL 机制就绑在请求头上，决定谁能读写哪个 topic。

数据层面要防敏感信息裸奔，可以在 payload 内部做字段级加密，像支付类消息里的卡号，别指望网络层全兜住。审计和日志也得跟上，关键操作记录请求来源、时间、动作，出事能追溯。重放攻击靠 nonce 或时间戳窗口来防，每个请求带唯一序列号，服务端做校验去重。



其实很多开源框架比如 gRPC 或 Dubbo，都把这套模式揉进了自己的扩展点里。MCP 如果是自研协议，这些环节一个都不能少，否则高压环境下容易被钻空子。

如何将已有的应用转换成 MCP 服务？

MCP 本质上是一套面向云原生的模块化编程模型，核心是把传统应用拆解成可独立部署、按需加载的 **微内核 + 插件** 结构。你不需要重写整个系统，关键是识别出哪些模块具备“可插拔”特征。

一般适合改造的模块是那些业务边界清晰、依赖收敛、有明确接口定义的部分。比如你在用 Spring Boot 做网关，里面的鉴权、限流、日志收集这些功能就可以拆成 MCP 插件。它们共用一套上下文，但彼此不耦合。

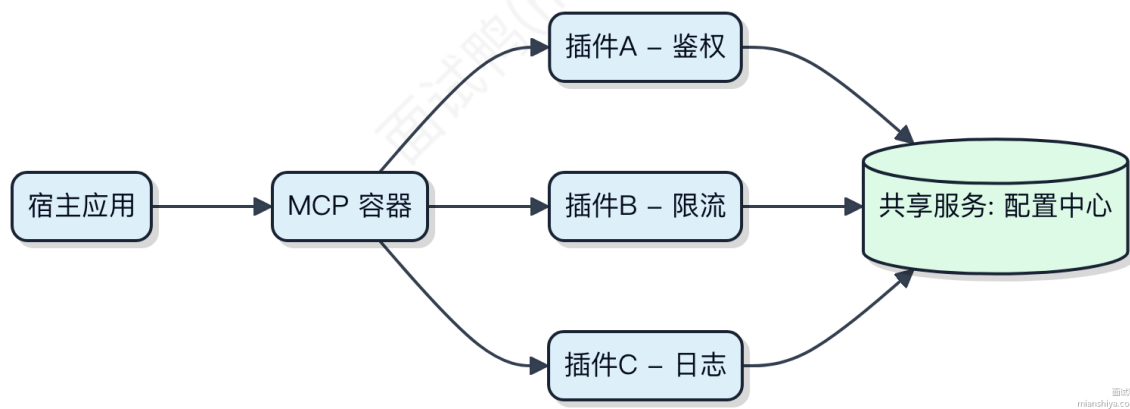
改造第一步是引入 MCP 运行时容器，比如基于 Jigsaw 模块系统或自研 ClassLoader 隔离机制。然后把原有功能打成 jar 包，配上 module.json 或注解声明入口类和依赖关系。启动时容器会按拓扑顺序加载并激活。

代码上最简单的做法是在原项目加一个启动器：

```
@McpModule(name = "auth-plugin", version = "1.0")
public class AuthPlugin implements ModuleLifecycle {
    public void onStart(Context ctx) {
        ctx.registerFilter(new AuthFilter());
    }
}
```

注意类加载器隔离，避免第三方库冲突。像 Apollo、Nacos 这类配置中心的客户端通常要下沉到宿主，插件通过统一 API 获取配置。

常见翻车点是静态变量共享和线程池滥用。不同插件如果都 new 自己的 ThreadPoolExecutor，很容易把机器打爆。建议统一用宿主提供的执行器。

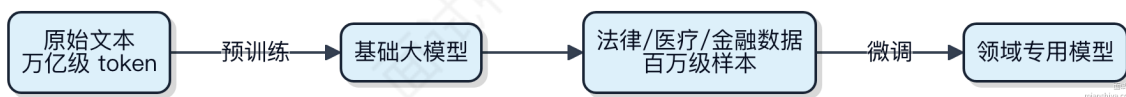


这种架构在阿里内部的 Sentinel 控制台、Ant Design Pro 后台都有落地，主要是为了支撑多租户定制化需求。小团队用的话，得评估好复杂度收益比，别为了架构而架构。

什么是大模型微调？与预训练的核心区别是什么？

大模型微调是在预训练好的模型基础上，用特定领域的数据继续训练，让它适应具体任务。比如你有个通用语言模型，想让它擅长写法律文书，就拿一堆法律文本去微调它。

- 1) 预训练是打基础，模型在海量无标注数据上学习语言规律，目标是理解“什么是语言”。这个阶段通常要用上千张GPU卡跑几周，参数量动不动上百亿。
- 2) 微调是定向优化，用的是小规模、有标注的领域数据，比如客服对话、医疗报告。训练轮次少，计算资源消耗低，一般几张GPU就能搞定。它的目标不是从零学语言，而是调整已有知识去解决“某个具体问题”。
- 3) 两者的区别不在代码实现，而在数据和目标。预训练像通识教育，什么都学；微调像职业培训，专精一项。Hugging Face 上很多 `bert-base-chinese` 是预训练模型，而 `bert-wm-ext` 就是在此基础上做了额外微调的版本。



微调压根不重新训练整个网络，只是在原有权重上做小幅更新。如果任务差异太大，比如从文本生成跳到图像识别，那微调就搞不定了，得换架构重来。

常见的微调任务有哪些？

微调任务其实就是在预训练模型基础上，针对特定场景做适配。最常见的几种类型，基本覆盖了大部分 NLP 需求。

1) 文本分类

把句子或段落分到预定义类别里。比如情感分析判断评论是正面还是负面，用 BERT 接一个分类头就行。

```
// 伪代码示意
model = BERT.from_pretrained("bert-base-uncased")
classifier = nn.Linear(768, num_labels) // 接在最后一层向量上
```

2) 命名实体识别 (NER)

找出文本里的实体，像人名、地名、组织名。一般在每个 token 上做序列标注，CRF 层常用来优化标签间转移逻辑。

3) 问答任务 (QA)

最典型的是 SQuAD 这类抽取式问答。模型要从给定段落里标出答案的起止位置。输入是“问题+段落”，输出两个指针。

4) 文本生成

比如摘要生成、翻译。用 T5 或 BART 这类 seq2seq 模型，输入原文，输出压缩后的摘要。训练时用交叉熵算 loss。

5) 句子对关系判断

判断两句话是什么关系。比如自然语言推理 (MNLI)，看第二句是支持、矛盾还是中立于第一句。STS 任务则是算语义相似度。

这些任务的数据格式和输出头不一样，但训练方式都差不多：冻结部分底层参数，放开顶层微调，学习率通常设得比较小， $3e-5$ 到 $5e-5$ 就够了。数据量一般几千到几万条就能见效，**迁移学习**在这里真能省下大量训练成本。

常见的微调方法有哪些？

微调本质上是拿预训练模型在特定任务上做二次训练，让模型更适配具体场景。常见的方法选择得看数据量、算力和任务复杂度。

1) 全量微调就是把整个模型的参数都放开训练。效果通常最好，但显存消耗大，8 张 A100 都不一定扛住 Llama 2 7B 的全量微调。适合数据质量高、预算充足的场景。

2) **LoRA**（低秩自适应）现在成了主流方案。它冻结原模型权重，在旁边加个小的低秩矩阵做增量，训练时只更新这部分。显存能省下 70%，训练速度也快，Hugging Face 上大部分开源微调项目都在用。

```
# LoRA 核心思想：A 下降维度，B 升维，中间做低秩分解
lora_weights = (x @ A) @ B
output = original_output + lora_weights
```

3) Adapter 方法是在 Transformer 层之间插入小的神经网络模块，训练时固定主干，只调这些小模块。结构改动比 LoRA 大，但现在用得少一些。

4) P-Tuning 和 Prompt Tuning 属于提示微调，不改模型本身，而是学一组虚拟 token 的 embedding。适合少样本场景，但效果波动大，对初始化敏感。

选哪个得权衡。数据多、资源足，可以试全量微调；大多数情况下，LoRA 是性价比之选，像 Qwen、ChatGLM 的社区微调基本都靠它打底。

什么是护栏技术？

护栏技术本质是一种在系统运行时动态控制流量、防止雪崩的保护机制。它不追求彻底解决问题，而是快速止损，让系统能继续对外提供部分服务。

1) 核心思路是“宁可限流，不可拖垮”。比如一个电商系统的下单接口突然变慢，大量请求堆积，数据库连接被打满，这时候如果不加干预，整个应用可能直接宕机。护栏技术就是在这个临界点介入，把超出处理能力的请求直接拒绝或降级。

2) 常见的实现方式有熔断、限流、降级。熔断像电路保险丝，当错误率超过 50% 就直接切断请求，避免连锁故障；限流控制每秒最多放行 1000 个请求，多余的就地拦截；降级则是返回兜底数据，比如商品详情页的推荐模块挂了，就直接返回空列表，保证主流程可用。

3) 实际项目里，**Sentinel** 和 **Hystrix** 都是典型的护栏工具。拿 Sentinel 来说，你可以给某个接口配置 QPS 超过 50 就自动限流，规则可以实时推送，不用重启服务。

```
// 定义资源
SphU.entry("getOrder")
// 业务逻辑
```

这种机制特别适合用在高并发场景，比如秒杀系统。你不可能靠堆机器扛住所有流量，得靠这些“安全策略”来兜底。真正的稳定性不是不出问题，而是出问题时系统还能活着。

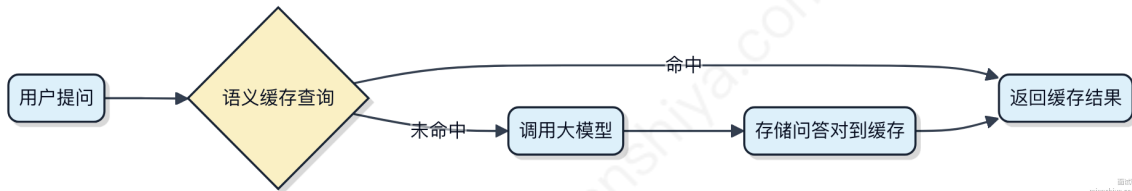
什么是 GPTCache？

GPTCache 是一个专为大模型应用设计的语义缓存中间件，目标是把重复或语义相近的请求直接拦截下来，减少对大模型 API 的调用次数，从而降成本、提响应速度。

它的核心思路不是简单地做字符串匹配，而是通过**向量相似度**来判断新来的请求和历史请求是否“意思差不多”。比如用户问“今天天气怎么样”和“现在外面晴吗”，文字不同但意图接近，GPTCache 就可能命中缓存。

整个流程一般是这样：

- 1) 用户请求进来，先被转换成 embedding 向量
- 2) 在向量数据库里搜最近似的几个结果
- 3) 如果相似度超过阈值，就返回缓存里的答案；否则走正常链路调大模型，并把新的 question-answer 存进缓存



它适合用在高频问答、客服机器人、RAG 系统这类场景，尤其是请求重复率高、对延迟敏感的应用。像 LangChain、LlamaIndex 这些框架都能集成 GPTCache。

不过要注意，embedding 模型的选择和距离计算方式会直接影响命中精度，太松容易误中，太严又起不到缓存作用。一般建议搭配 Milvus、Pinecone 或 Chroma 做向量存储，性能和扩展性更稳。

什么是 GPT Structured Outputs?

GPT Structured Outputs 是一种让大模型输出**结构化数据**的能力，比如 JSON 格式的结果，而不是自由文本。这在需要程序直接解析模型输出的场景里特别有用。

你告诉模型你想要的 schema，它就会按这个格式返回数据。这样一来，前端或后端代码就能直接消费输出，不用再写一堆正则或逻辑去提取信息。

比如你在调用 GPT 时启用 `response_format` 参数，并指定为 JSON Schema，模型就会严格按字段输出。OpenAI 从 2023 年底开始支持这个功能，适用于像 gpt-3.5-turbo 和 gpt-4 这样的模型。

```
{
  "name": "Alice",
  "age": 30,
  "city": "Beijing"
}
```

这种机制适合做配置生成、表单填充、API 数据抽取等任务。和以前“靠运气”解析文本不同，现在结果可预测、可验证。

不过得注意，模型不会自动知道什么时候该输出结构，必须显式设置参数，否则还是走自由生成。另外复杂 schema 可能增加 token 消耗，设计时要权衡清晰度与成本。

如果原题解中的某句话使用了超链接，不要对这句话做任何改动。

什么是 LangChain?

LangChain 是个帮助开发者快速搭建应用的框架，重点是让大模型和外部系统打通。你不用从零写 prompt 调用、上下文管理、工具调用这些重复代码，它都给你封装好了。

它的核心能力之一是**链式调用**，你可以把多个步骤串起来，比如先查数据库，再让大模型总结，最后发个通知。每个环节它都抽象成组件，像积木一样拼接。

另一个关键点是跟外部数据对接。比如你要做个客服机器人，不能光靠模型内置知识，得让它查你的产品文档。LangChain 提供了和向量数据库集成的能力，像和 Pinecone、Weaviate 对接，做检索增强生成（RAG），这样回答就有据可依。

它还支持 Agent 模式，就是让模型自己决定下一步干什么。比如用户问“昨天销售额最高的订单是谁”，模型可以自己判断要先查订单表，再查客户信息，然后组织语言回复。背后其实是模型输出特定格式指令，框架负责解析并执行。

代码上它很简洁，几行就能定义一个链：

```
from langchain.chains import LLMChain
chain = LLMChain(llm=llm, prompt=prompt)
response = chain.run("中国的首都是哪里? ")
```

适合做需要动态输入、多步推理、结合私有数据的场景，比如智能客服、数据分析助手。但如果你只是简单问答，可能显得重了。

什么是 LangGraph ?

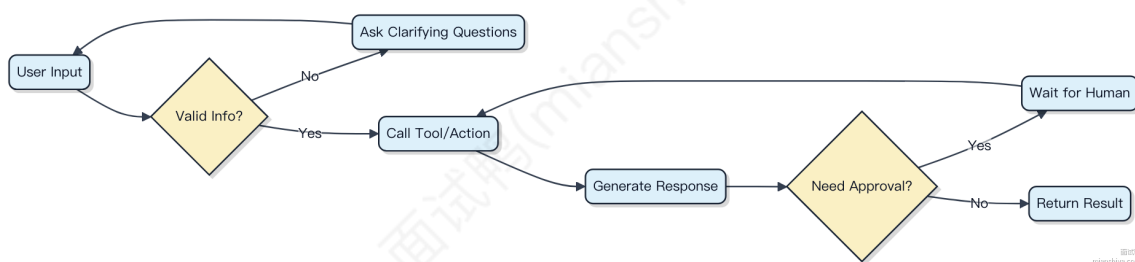
LangGraph 是基于 LangChain 构建的**有状态**的多智能体协作框架。它把多个 AI Agent 的交互过程建模成“图”（Graph），每个节点是一个执行单元，边代表控制流或数据流向。

这种图结构让你能清晰定义复杂的 agent 协作逻辑。比如一个节点是用户输入处理，另一个是工具调用，第三个是决策判断，通过边连接它们的执行顺序。整个流程的状态被显式维护，后续节点可以访问前面所有步骤的数据。

它特别适合做需要**反复来回交互**的任务。像客服系统里，一个 agent 处理意图识别，发现信息不全就转给追问 agent，补全后再交给业务办理 agent。这种循环、条件跳转、状态保持的场景，传统流水线搞不定，而 LangGraph 能扛住。

代码上其实就是定义一堆节点函数，然后用 `add_node` 和 `add_edge` 把它们串起来，最后编译成一个可执行的 graph。调试时还能看到每一步的状态快照，排查问题方便多了。

举个实际场景：用 LangGraph 搭一个多轮审批流，结合 Human-in-the-loop，在关键决策点暂停等待人工确认，确认后继续往下走，整个过程可追溯。



这个模型让复杂 agent 工作流变得结构化，而不是一堆混乱的回调和全局变量。

LangGraph 编排的原理是什么？

LangGraph 的本质是把大模型的执行过程变成一个有向图，每个节点是一个动作或决策点，边代表执行路径。你给它一个输入，它就从起点开始，按图的结构一步步走，走到哪、下一步干啥，由当前节点的逻辑决定。

1) 节点可以是调用大模型、执行工具、做条件判断，甚至是人工审核这种阻塞操作。比如在客服机器人里，一个节点可能是“识别用户意图”，下一个节点根据意图跳转到“查订单”或者“投诉处理”。

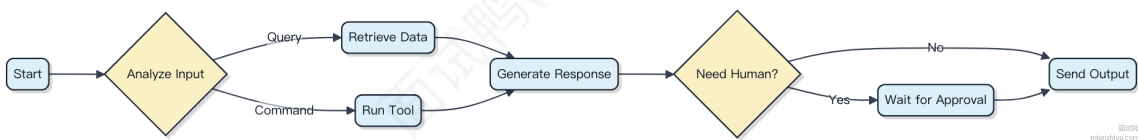
2) 每一步的状态都通过一个共享的 state 对象传递, 这个 state 是可变的, 后续节点能读也能改。这就让整个流程有了上下文记忆, 不像普通函数调用那样孤立。

3) 控制流靠的是 conditional edges, 也就是条件边。你可以写个函数判断 state 里的字段, 返回下一个节点的名字。比如判断 “是否需要人工介入”, 返回 "human_in_the_loop" 或 "continue_automated"。

代码上其实就是定义节点函数和边规则:

```
graph.add_node("generate", generate)
graph.add_conditional_edges("generate", route_generate_result)
```

整个执行过程是同步推进的, 但每步之间可以暂停、恢复, 适合长周期任务。像 AutoGen、CrewAI 这些框架底层也用了类似思路, 只是 LangGraph 更灵活, 支持复杂 DAG。



LangChain 和 LangGraph 有什么区别?

LangChain 是个工具箱, 帮你快速搭起和大模型交互的流水线。你拿它做点简单的链式调用, 比如文档加载、切分、检索再喂给 LLM 生成答案, 整个流程像一条直线, 串几个组件就跑起来。适合做些原型或者轻量级应用, 比如基于文档问答的小工具。

LangGraph 就不一样了, 它是专门搞**有状态、多步骤、带循环**的复杂 Agent 流程的。你可以把它看成是 LangChain 的“升级控制器”。它用**有向图**来建模流程, 节点是动作, 边是转移逻辑, 能表达“失败重试”、“条件跳转”、“人类介入”这种复杂控制流。比如你做个多智能体协作系统, A 干完活交给 B, B 觉得不行又回滚给 A, 这种环状依赖, LangChain 原生链式结构压根不经过, 但 LangGraph 天生支持。

打个比方: LangChain 是条流水线, 工件从头走到尾; LangGraph 是个调度网络, 能根据情况动态派单、回流、并行处理。

代码上, LangGraph 核心是 StateGraph, 你定义一个共享状态, 每个节点函数读写这个状态, 然后通过条件边决定下一步去哪:

```
from langgraph.graph import StateGraph

graph = StateGraph(MyState)
graph.add_node("draft", draft_node)
graph.add_node("review", review_node)
graph.add_conditional_edges("review", should_rework)
graph.add_edge("draft", "review")
```

这种模式在做 AutoGen、MetaGPT 那类复杂 Agent 编排时特别扛住, 脏活累活都让它揽了。

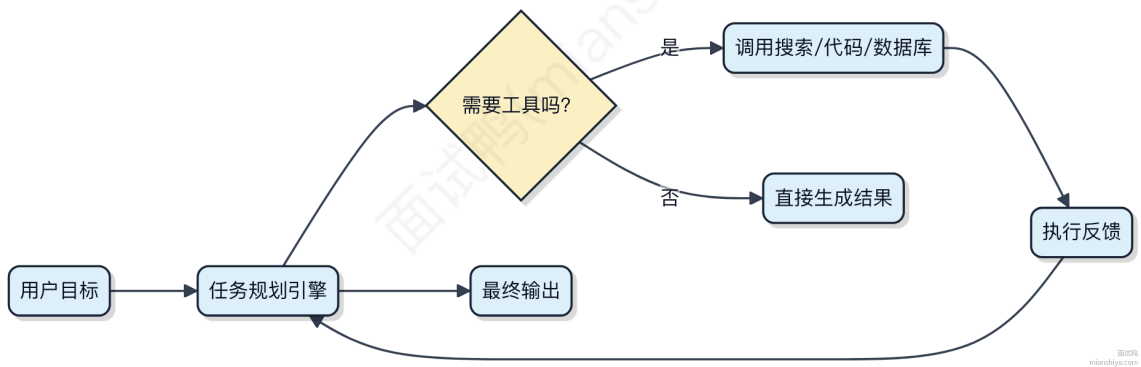
什么是 Manus? 说说你对它的了解

Manus 不是 Java 或中间件领域的技术组件, 它其实是一个由上海 AI 实验室推出的开源大模型智能体框架。你可以把它理解成一个能让大语言模型“动起来”的工具链, 让模型不仅能回答问题, 还能执行任务。

它最大的特点是支持**自主规划与工具调用**。比如给它一个复杂目标，像“分析某支股票的历史走势并生成报告”，Manus 能自己拆解任务：先搜索实时股价，再调用 Python 分析数据，最后生成可视化图表和文字总结。背后依赖的是强化学习和思维链（CoT）的结合。

这类框架一般会集成常见工具接口，比如数据库连接、API 调用、代码解释器等。你在用的时候只需要定义目标，剩下的步骤由 agent 自主决策。类似项目还有 LangChain + AutoGPT 的组合，但 Manus 更强调端到端的自治能力和中文场景优化。

典型架构包含几个核心模块：任务分解器、工具路由、记忆存储和反馈循环。它不像传统微服务那样靠 RPC 通信，而是通过语义驱动动作选择。



目前适用于自动化数据分析、智能运维助手等场景。如果你在做 AIGC 相关系统集成，可以考虑用它快速搭建 agent 流程，但生产环境还得注意结果可解释性和执行超时控制。

ReAct 是什么？说说它的原理

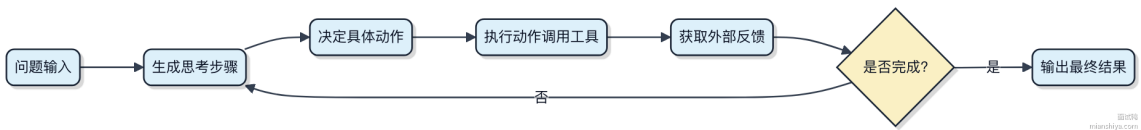
ReAct 是一种让大模型在做推理时，既能思考（Reasoning）又能行动（Action）的框架。它的核心不是单纯输出答案，而是通过交错的“想一步、做一步”来完成复杂任务。

模型在处理问题时，会先生成一段思考内容，比如分析当前情况、设定下一步目标，然后决定一个具体操作，比如调用工具、查数据库或搜索网页。这个动作执行后拿到结果，再喂回模型继续下一轮思考，直到得出最终答案。

这种模式特别适合需要外部交互的任务，像客服机器人查订单状态，或者智能助手执行多步操作。相比纯推理，它能获取实时数据；相比纯指令驱动，它多了规划和反思能力。

举个例子，你要查天气并决定是否带伞，ReAct 会先想“我需要知道当前位置的天气”，然后触发“调用天气API”动作，拿到结果后再判断“预报有雨，建议带伞”。

整个过程就像人在解决问题时的习惯：思考 → 行动 → 观察 → 再思考。语言模型不再只是被动回答，而是主动推进任务。



LLM Agent 的基本架构有哪些组成部分？

LLM Agent 不是简单地调用大模型，而是一个能感知环境、做出决策并执行动作的智能体。它的结构更像是一个闭环控制系统。

1) 感知模块 (Perception)

负责接收外部输入，比如用户指令、环境状态或历史交互记录。这部分会把原始信息转换成模型可理解的格式，通常就是拼接到 prompt 里。

2) 规划模块 (Planning)

这是 Agent 的“大脑”，又可以拆成两块：

- **推理 (Reasoning)**：让模型对当前问题进行多步思考，比如使用思维链 (Chain-of-Thought)，先想“我该怎么做”，再决定下一步。
- **任务分解 (Task Decomposition)**：复杂任务会被拆成多个子任务，按序或并行处理，像 LangChain 中的 Plan-and-Execute 就是典型实现。

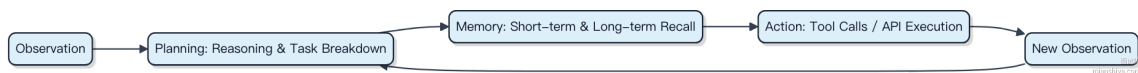
3) 记忆模块 (Memory)

维持短期和长期上下文。短期记忆一般是当前对话的 token 上下文窗口，受限於模型长度；长期记忆则依赖向量数据库，比如用 Milvus 或 Chroma 存储历史经验，需要时检索召回。

4) 工具调用与执行 (Tool Use & Action)

Agent 能主动调用外部工具，比如查天气、执行代码、操作数据库。主流做法是通过函数调用 (Function Calling) 机制，让模型输出结构化 JSON 去触发 API，例如在 OpenAI 的接口中定义 tools 列表。

整个流程走下来，其实就是一个“观察 → 思考 → 决策 → 行动 → 观察”的循环。



LLM Agent 常见功能有哪些？

LLM Agent 不是简单的问答机器人，它得能感知环境、做规划、调工具、持续执行直到目标完成。你可以把它看成一个“AI 执行官”，自己拆任务、调接口、纠错重试。

1) 规划与目标分解

Agent 接到“分析竞品并写报告”这种大任务，不会一口吞，而是拆成查资料、读 PDF、对比数据、生成初稿、润色几步。像 LangChain 的 PlanAndExecute 就是干这个的典型模式。

2) 工具调用 (Function Calling)

它得知道什么时候该让别人干活。比如查实时股价，直接调 Alpha Vantage API；订会议室就走钉钉或飞书开放平台。模型输出结构化指令，框架负责转成真实调用。

3) 记忆能力

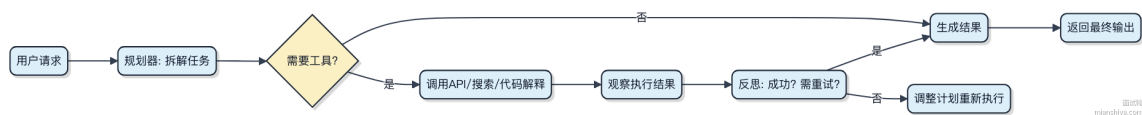
短时记忆靠上下文窗口存对话历史，长时记忆就得接外部存储。比如用 Chroma 或 Milvus 存过往决策记录，下次遇到类似任务能参考。

4) 自主执行与反思

有些 Agent 能自己决定下一步。AutoGPT 是个例子，但它容易陷入循环。更稳的做法是 ReAct 模式：每步“思考-行动-观察”，像人一样边做边看结果。

5) 多 Agent 协作 (进阶)

复杂场景下会分工。比如一个当产品经理提需求，一个当工程师写代码，一个当测试员验证，通过内部通信达成目标。MetaGPT 就模拟了这种角色分工。



别指望它百分百靠谱，超长上下文推理还是会乱，工具调用也可能失败。关键是设计好 fallback 机制和人工介入点。

市面上有哪些主流的 LLM Agent 框架？各自的特点是什么？

主流的 LLM Agent 框架这几年发展很快，基本都围绕“如何让大模型更可靠地完成复杂任务”在设计。

1) LangChain 是最早火起来的框架，生态最全，支持的数据源、工具、模型最多。它的优势在于灵活，像搭积木一样组合组件，适合做原型验证。但灵活性带来的问题是代码结构容易混乱，调试困难，生产环境需要较强的工程能力压得住。

2) LlamaIndex 起家是为了解决大模型对接私有数据的问题，核心强项是**检索增强生成**（RAG）。它对文档切分、索引构建、查询优化做了很多深度优化，比如支持分层索引、动态获取上下文。如果你的 Agent 主要靠读文档回答问题，比如用在企业知识库场景，LlamaIndex 通常比 LangChain 更高效。

3) AutoGPT 名气很大，主打“自主任务分解”，能自己拆目标、调工具、循环执行。理想很丰满，但实际跑起来经常陷入无效循环，输出不可控，目前更多是概念验证，生产环境基本不用。

4) Semantic Kernel 是微软出的，和 Azure 生态深度绑定，适合 .NET 技术栈的团队。它强调“规划器”概念，把任务拆解做成可配置模块，和 LangChain 类似但更重企业级控制。

5) Dify 和 FastGPT 属于低代码平台型框架，把 Agent 的流程可视化，拖拽就能编排 prompt、工具和逻辑。适合产品经理或业务方快速试错，但灵活性受限，复杂逻辑还得写代码。

总的来说，选哪个框架看场景。做 RAG 优先看 LlamaIndex，要快速搭 demo 用 LangChain 或 Dify，企业级可控流程可以看看 Semantic Kernel。

什么是 LangChain Agent?

LangChain Agent 的本质是一个能做决策的“大脑”，它不直接生成最终结果，而是根据目标动态规划步骤，调用工具去执行。

1) Agent 会接收用户输入，结合预设的提示词和可用工具描述，决定下一步动作。这个动作要么是调用某个工具（比如搜索、数据库查询），要么是直接返回结果。

2) 它依赖 **LLM** 做推理判断。比如你问“今天上海天气怎么样”，Agent 会先意识到需要实时数据，于是选择调用天气 API 工具，拿到结果后再让 LLM 汇总成自然语言回复。

3) 工具（Tools）是关键组件，每个工具对应一个函数，声明了名称、描述和入参。Agent 不直接写代码，而是通过描述理解工具能力。你可以接入 SerpAPI 做搜索，或者自定义一个查订单的函数。

```
// 伪代码示意：定义一个搜索工具
Tool searchTool = Tool.builder()
    .name("search")
    .description("用于查询实时信息，比如天气、新闻")
    .function(query -> callSerpAPI(query))
    .build();
```

和普通 Chain 的区别在于，Chain 是固定流程，Agent 是动态路径。你没法提前预知它会怎么走，就像你不能预测人思考的过程。

适合场景：需要多步推理、外部数据获取、动态决策的任务。但如果逻辑简单，直接写 Chain 更高效，别为了用而用。

什么是 LangChain model?

LangChain model 并不是一个独立的模型，它其实是 LangChain 框架中对各种大语言模型（LLM）的统一抽象封装。你用它来调用 GPT、通义千问、ChatGLM 这些模型时，接口都是一致的。

- 1) 它的核心是提供一个标准接口，让开发者不用关心底层模型是哪家的，都能用同样的方式传入 prompt、拿到输出。比如你切换从 OpenAI 到本地部署的 Llama，代码改动可以非常小。
- 2) 除了基础的文本生成，它还支持流式输出、带历史对话的会话模型（ChatModel）、以及结构化输出解析。像和用户多轮聊天的场景，直接用 RunnableWithMessageHistory 就能管理上下文。
- 3) 实际开发里，你经常要拼接提示词、调用工具、做数据预处理。LangChain model 和 PromptTemplate、RetrievalChain 这些组件能无缝配合，把这一整条链路串起来。

举个简单例子：

```
// 伪 Java 风格代码示意
ChatModel model = new ChatOpenAI("gpt-3.5-turbo");
String response = model.call("请用 Java 写个单例模式");
```

它真正解决的是 AI 应用开发中的胶水问题。没有它的时候，每个模型调用都要写一堆适配代码，现在这些脏活累活框架帮你扛住了。

A2A 协议的工作原理是怎样的？

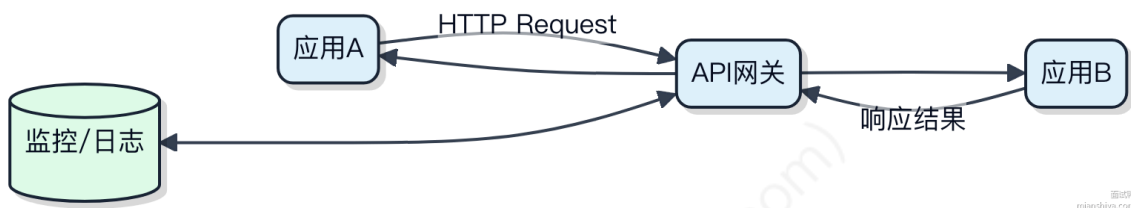
A2A（Application-to-Application）协议不是某个具体的技术标准，而是一种通信范式，指的是应用系统之间通过定义好的接口直接交互。这种模式在微服务、中台架构里特别常见。

- 1) 通常基于 HTTP/HTTPS 或消息队列实现，比如两个服务间用 RESTful API 交换 JSON 数据，或者通过 Kafka 异步传递事件。关键在于双方约定好数据格式和调用语义。
- 2) 安全控制很重要，一般会引入 OAuth2、JWT 做身份鉴权。比如一个订单系统调用库存系统时，必须携带有效 token，对方验证通过才执行扣减操作。
- 3) 为了降低耦合，很多公司会加一层 API 网关统一管理 A2A 流量。像阿里云的 API Gateway 就能做限流、监控、访问控制，后端服务只专注业务逻辑。

代码上其实很简单，就是一个服务发起请求，另一个提供接口：

```
// 消费方调用远程服务
ResponseEntity<StockResult> result = restTemplate.getForEntity(
    "http://stock-service/decrease?itemId=1001",
    StockResult.class
);
```

但实际落地要考虑超时、重试、熔断这些容错机制，不然一个服务卡住可能拖垮整条链路。Spring Cloud OpenFeign + Hystrix 是一种典型组合。



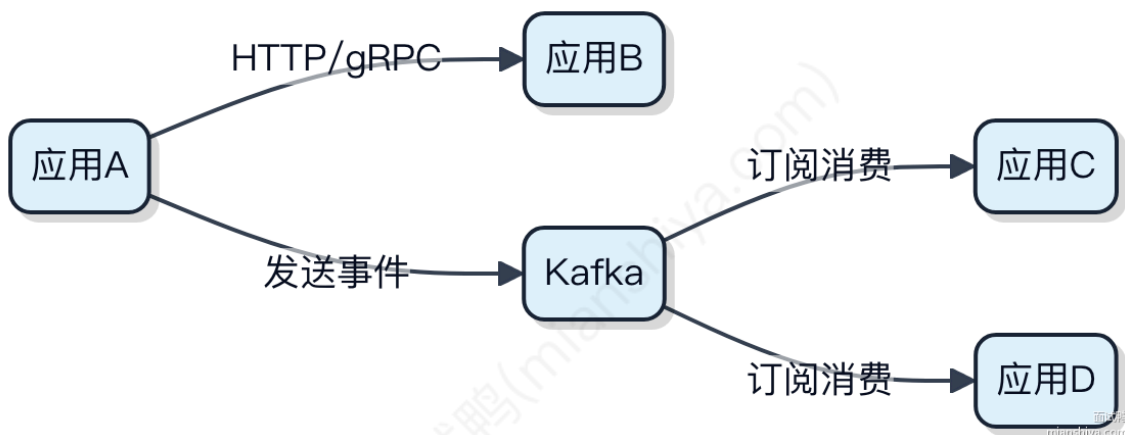
A2A 协议的工作流程是怎样的？

A2A（Application-to-Application）协议不是某个具体的标准协议，而是一类系统间通信模式的统称。它指的是两个应用程序直接通过 API、消息或文件等方式交换数据，不依赖人工干预。

这类通信一般走服务间调用链路，常见于微服务架构。比如订单系统调用库存系统扣减接口，这就是典型的 A2A 场景。整个流程从发起方构造请求开始，经过网络传输，到达接收方处理并返回结果。

- 1) 发起方应用准备好业务参数，封装成请求体
- 2) 通过 HTTP/REST 或 gRPC 等协议发送到目标服务
- 3) 目标服务接收到请求后解析参数，执行业务逻辑
- 4) 将结果序列化为响应体返回给调用方
- 5) 发起方根据响应做后续处理，如重试、落库或通知下游

如果是异步场景，会引入消息中间件。比如用户注册后需要发邮件、加积分，主流程写入消息队列，由消费者分别调用邮件服务和积分服务。这种情况下 A2A 的流转就变成了事件驱动，Kafka 或 RocketMQ 扮演了关键角色。



要注意的是，A2A 流量通常比用户请求更稳定，但对准确性和幂等性要求更高。一旦出错可能引发数据不一致，所以得配合熔断、限流、trace 跟踪这些机制来保障可靠性。

大模型的结构化输出指的是什么？

大模型的结构化输出，说白了就是让模型不光吐文字，还能按指定格式返回数据，比如 JSON、XML 或特定 schema 的内容。你给个指令，它能生成符合接口契约的响应，而不是一堆自由发挥的自然语言。

这在实际系统集成里特别关键。比如你让大模型从一段文本里抽用户信息，直接回“张三，35岁，北京”还得再解析，但如果要求结构化输出，它就能直接给你：

```
{
  "name": "张三",
  "age": 35,
  "city": "北京"
}
```

这种能力一般靠提示词工程（prompt engineering）引导，或者用像 **JSON mode** 这类机制强制约束输出格式。OpenAI 的 API 支持 `response_format={ "type": "json_object" }`，这时候模型就必须输出合法 JSON，否则调用方解析要出问题。

更进一步的做法是结合 **function calling** 或 **tool use**，把结构化输出当成调用外部工具的中间协议。比如你让模型判断是否需要查天气，它就返回一个带 `function_call` 字段的 JSON，里面是函数名和参数，下游系统直接执行。

这类技术减少了后处理成本，让大模型能真正嵌入到软件流程里，而不是只当个聊天玩具。不过对模型本身的推理一致性和格式遵从度要求更高，小模型容易格式错乱，得反复校验。

什么是 Google ADK?

Google ADK（Android Debug Bridge）这个说法其实是个常见的误解。你可能想问的是 **ADB**，也就是 Android Debug Bridge。

ADB 是 Android 开发里最基础的调试工具，它让你能在电脑上通过命令行跟手机通信。开发或者测试时，装应用、看日志、执行 shell 命令都靠它。

它由三部分组成：1) 运行在电脑上的 **adb 客户端** 2) 运行在手机上的 **adb 服务** 3) 中间通过 USB 或网络连接

比如你想把一个 APK 安装到设备，直接敲：

```
adb install app-debug.apk
```

或者进 shell 看文件：

```
adb shell
ls /sdcard/
```

还有一种情况是 **ADK**，全称是 Android Open Accessory Development Kit，这是 Google 提供的一套让 Android 设备与外部硬件配件通信的方案，基于 USB 主机模式。像一些工业设备、外接传感器通过 USB 连手机，会用到它。不过实际项目中用得不多，大部分场景被蓝牙或 Wi-Fi 取代了。

简单说，日常开发提到的“ADK”九成都是口误，真正要用的是 ADB。遇到真配件通信需求，才会碰得到 ADK 那套协议和权限声明。

解释LangChain框架中的Chain和Agent概念，并举例说明各自的应用场景

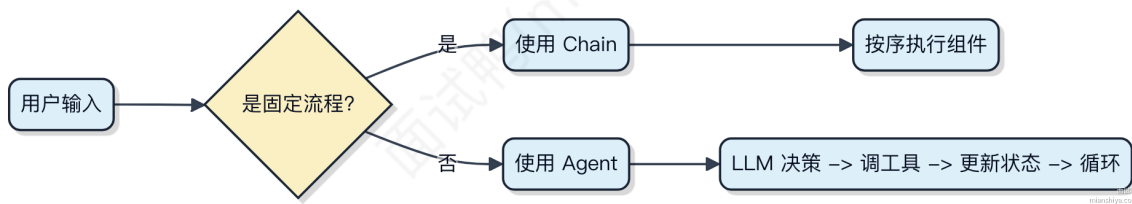
LangChain 里的 Chain 和 Agent 是两个关键抽象，解决的是不同复杂度的任务编排问题。

Chain 其实就是一个函数式的组合，把多个处理步骤像链条一样串起来。每个环节的输出自动作为下一个环节的输入，中间可以穿插对 LLM 的调用、数据处理或外部工具。比如你做客服机器人，用户问“今天北京天气怎么样”，你可以先用一个 PromptTemplate 构造查询语句，接着调用 LLM 解析出地点和需求，再交给一个 WeatherTool 获取真实数据，最后用另一个模板生成回复。这个流程就是典型的 Chain，**顺序执行**，逻辑固定。

```
// 伪代码示意
String result = chain.execute("今天北京天气如何? ");
```

而 Agent 更灵活，它让 LLM 自己决定“下一步干什么”。LLM 不只是回答问题，而是充当“决策大脑”，根据当前状态选择调用哪个工具、是否需要查数据库、要不要反复推理。比如做一个智能数据分析助手，用户说“帮我分析上周销售额下降的原因”，Agent 可能先查销售表，再对比营销活动记录，发现某区域广告暂停后销量下滑，于是建议检查推广策略。整个过程路径不固定，由 LLM 动态规划。

- 1) Chain 适合流程明确、步骤固定的场景，比如内容生成流水线、结构化信息提取
- 2) Agent 适合开放性任务，需要动态决策，比如智能客服、自动化运维诊断
- 3) Agent 的代价是性能开销大，因为每一步都要走 LLM 推理，容易陷入循环或误判



什么是大模型的“涌现能力”? 列举三种典型表现并解释其可能成因

大模型的“涌现能力”指的是当模型参数规模达到某个临界点后，突然展现出小模型不具备的能力，这种能力无法通过简单外推得到。

1) 上下文学习 (In-Context Learning)

模型仅凭输入中的几个示例就能完成新任务，不需要额外微调。比如给 GPT-3 输入“苹果→水果，胡萝卜→蔬菜，番茄→?” 它能推理出“蔬菜”。这背后其实是模型在推理时动态构建了内部计算路径，把提示词当成了临时指令模板。

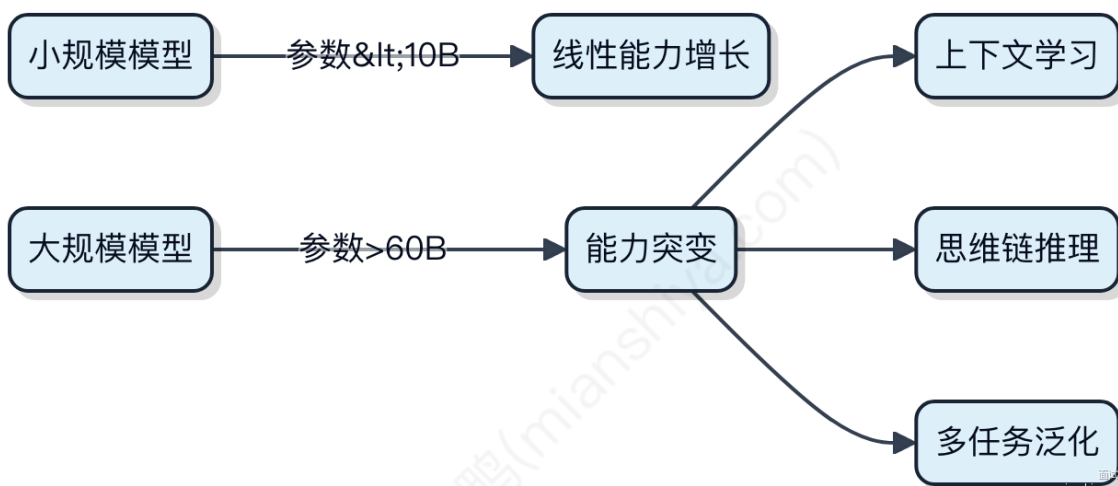
2) 思维链推理 (Chain-of-Thought Reasoning)

模型能输出中间推理步骤，比如数学题先列式再求解。实验发现，只要在提示中加入“让我们一步步思考”，PaLM、GPT-3.5 等模型就能激活这种行为。这说明大模型内部已经形成了类似推理的计算模式，但需要外部触发。

3) 多任务泛化

同一个模型可以同时处理翻译、代码生成、逻辑判断等完全不同的任务，且表现接近专用模型。像 Llama 系列在训练时混入了海量多源数据，使得其权重空间自然分化出多个功能子空间。

可能成因在于：参数量突破临界点（一般认为要超 60B）后，模型表达能力发生质变，高维空间中出现新的结构化组织方式。训练数据的多样性让模型学会了“元学习”——不是记答案，而是学怎么学。



假设需要让大模型生成一个React表单组件代码，请设计一个包含上下文约束的Prompt（需包含数据验证、错误提示等要求）

你让大模型写 React 表单，光说“写个表单”它肯定给你个最简单的 input 加按钮。要拿到能落地的代码，得把约束条件一次性给全。

关键是在 Prompt 里明确结构、行为和边界。比如你要一个用户注册表单，就得指定字段、验证规则、错误提示方式、提交逻辑。

可以这样组织上下文：

- 1) 表单包含用户名、邮箱、密码、确认密码四个字段
- 2) 所有字段必填，用户名至少 3 字符，邮箱格式校验，密码需 8 位以上且含大小写和数字
- 3) 实时校验：输入时显示错误信息，不满足规则时禁用提交按钮
- 4) 错误提示使用 React Hook Form 的 `errors` 对象驱动，配合 `Zod` 做 schema 校验
- 5) 样式用 `className` 预留，不依赖具体 UI 库

代码示例核心结构：

```
const schema = z.object({
  username: z.string().min(3),
  email: z.string().email(),
  password: z.string().min(8).regex(/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)/),
  confirmPassword: z.string()
}).refine(data => data.password === data.confirmPassword, {
  message: "Passwords don't match",
  path: ["confirmPassword"]
});
```

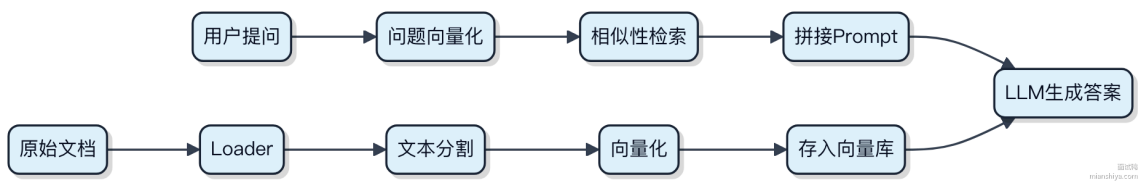
搭配 `useForm` 和 `zodResolver`，把 `register` 绑到输入框，`errors` 渲染提示，按钮根据 `formState.isValid` 控制是否可点。

这种 Prompt 能让模型输出接近生产环境的代码，而不是玩具示例。你给的约束越贴近实际项目，生成结果越能直接用。

请描述使用LangChain构建一个文档问答系统的关键技术组件及实现步骤

构建文档问答系统，本质是把非结构化文本转成向量，再结合大模型做自然语言交互。LangChain 提供了一整套工具链来简化这个过程。

- 1) 文档加载与分割
先用 DocumentLoaders 从 PDF、Word 或网页抓取原始内容。拿到文本后得切分，一般按段落或固定长度（比如 500 token），太长会超出模型上下文，太短又丢失语义。这里常用 RecursiveCharacterTextSplitter。
- 2) 向量化与检索
切好的文本块喂给嵌入模型（Embedding Model），比如 OpenAI 的 text-embedding-ada-002 或开源的 BGE。生成的向量存进向量数据库，像 Chroma、Pinecone 或 Milvus。用户提问时，系统先把问题向量化，去库里找最相似的 top-k 文本块，这叫**相关性检索**。
- 3) 提示工程与模型调用
检索到的上下文拼上用户问题，组装成 prompt，丢给大语言模型（LLM）。LangChain 提供 PromptTemplate 来规范格式，避免模型胡说。典型的模板就是“根据以下内容回答问题：{context} 问题：{question}”。
- 4) 链式调用（Chain）
把上述步骤串起来用 LLMChain 或 RetrievalQA 链。RetrievalQA 是现成的高级封装，一行代码就能连起检索器和 LLM，适合快速搭建原型。

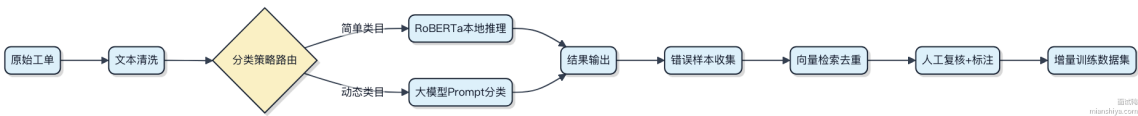


整个流程里，数据预处理和检索质量直接决定效果上限。模型再强，喂进去的上下文不对，也搞不定准确回答。

假设要开发一个智能工单分类系统，请拆解AI可参与的环节并说明技术选型思路

智能工单分类不是简单打个标签，得把AI嵌进整个流转链条里，哪里能省人力、提准确率就往哪插。

- 1) 工单预处理阶段，文本清洗和标准化可以用规则+轻量模型。比如用户输入“登不录”，先用正则和纠错词典做基础清洗，再上一个 TinyBERT 做语义对齐，转成标准表述“无法登录系统”。这步别上大模型，响应延迟扛不住。
- 2) 核心分类环节看数据量和类目复杂度。如果只有几十个固定类别，RoBERTa + CRF 足够，推理快、好维护。要是类目动态增长，比如每天新增业务线，就得考虑 Prompt Learning 配合 P-tuning v2，接通大模型如 ChatGLM-6B 做 zero-shot 分类，类目一改prompt跟着变，不用重新训练。
- 3) 后置反馈闭环必须做。分错的工单要进人工复核池，这部分数据定期回流做增量训练。可以搭个轻量 Label Studio + Milvus 相似样本检索，自动推荐历史相似案例给标注员，效率翻倍。
- 4) 工程部署上，小模型走 TensorFlow Lite 或 ONNX Runtime 嵌入服务内部，大模型用 vLLM 做批处理异步推理。别把所有请求都怼到大模型上，成本直接炸。



当需要处理超长上下文窗口限制时，有哪些可行的工程解决方案？

处理超长上下文，本质是绕过或优化模型的 token 长度硬限制。直接喂满动辄 32K、128K 上下文，成本高且效果不一定好，得靠工程手段拆解。

1) 分块与索引检索

把长文档切片存入向量数据库，比如用 LangChain + Chroma/Pinecone，查询时只召回相关 chunk。关键在分块策略，按段落切可能割裂语义，结合滑动窗口重叠（overlap）能缓解。但太碎了会丢失全局逻辑。

2) 摘要增强（Summary Augmentation）

对历史对话或文档提前生成摘要，新请求到来时，把摘要当上下文注入。像 LLM-powered summarizer 在对话系统里很常见，能把前 10 轮压缩成 1-2 句，省下大量 token。

3) 位置插值（Position Interpolation）

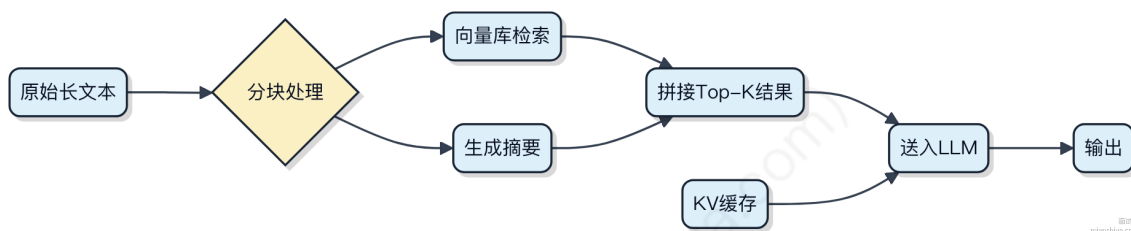
针对 Transformer 架构的位置编码做手脚。原生 RoPE 支持外推但衰减快，采用 NTK-by-parts 或 YaRN 等方法，在推理时动态调整位置频率，让模型“脑补”超出训练长度的位置信息。实测能在 32K 模型上跑 64K 输入，效果接近原生长上下文。

4) 侧边缓存（Side Cache）

Google 提出的思路，把已计算的 KV 缓存复用起来。比如用户修改了 prompt 中一小段，不需要全量 recompute，命中部分直接从 cache 读。适合交互式场景，降低延迟和算力消耗。

5) 分层上下文路由

复杂系统如 Agent 工作流，用调度层决定哪些信息进长期记忆（vector store），哪些放短期上下文。AutoGPT 类项目就这么干，核心状态放 context，细节查数据库。



当大模型API响应延迟超过1秒时，前端可以采取哪些优化策略保证用户体验？

前端能做的其实很有限，核心是“让用户感觉快”，而不是真的让模型变快。大模型API本身耗时主要在服务端推理，前端更多是体验层的兜底。

1) 请求发出去之后，立刻给用户反馈，别干等。比如马上展示一个“思考中...”的动画或者骨架屏，告诉用户系统已经在处理了，避免误操作重复提交。

2) 如果接口超过800ms还没回来，可以主动提示“生成时间较长，请稍候”。这种预期管理很重要，用户知道要等，就不会觉得卡死了。

3) 优先考虑流式响应（streaming）。让后端以SSE或WebSocket方式分段返回结果，前端拿到一段就渲染一段。就像 ChatGPT 那样逐字输出，视觉上比等1秒再刷出来舒服得多。

4) 结合loading状态和取消按钮。允许用户在等待时手动中断请求，提升控制感。特别是移动端，长时间无响应容易引发焦虑。

5) 缓存历史问答对。对于常见问题，比如产品FAQ，可以直接走本地缓存秒回，根本不用走大模型。

6) 降级策略。检测到连续超时，可以自动切换到轻量模型接口或静态回答模板，保证基础可用性。

使用LangChain时，如何实现多路召回结果的动态权重分配？

多路召回的动态权重分配，关键在于根据查询实时计算各路结果的置信度或相关性得分，而不是用固定权重拍脑袋。

1) 最直接的做法是引入一个重排序模型（Reranker），比如 BGE-Reranker 或 Cohere Rerank。把不同来源召回的候选集合并后，统一喂给 Reranker 打分，它会输出基于 query 的相关性排序，自然就实现了“动态加权”。LangChain 里可以用 `RunnableLambda` 包装 reranker 调用。

2) 另一种思路是基于查询特征做路由加权。比如用户问的是时间敏感问题，就提高来自新闻索引或时序数据库那一路的权重；如果是专业术语，就抬高知识图谱或维基数据源的分值。这在 LangChain 可以通过 `RouterRetriever` 实现，配合 LLM 判断 query 类型，动态选择或加权子检索器。

3) 还可以让 LLM 自己评估各路结果的相关性。把每路召回的 top-k 和 query 一起输入 LLM，让它打分或排序。虽然贵点慢点，但灵活性最高，适合对效果要求极高的场景。

代码上，核心是组合多个 `BaseRetriever` 并在外层做融合：

```
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever

ensemble_retriever = EnsembleRetriever(
    retrievers=[vector_retriever, BM25Retriever.from_texts(...)],
    weights=[0.6, 0.4] # 这里可以动态算出来再传
)
```

重点是 `weights` 别写死。你可以先跑个轻量模型预估各路匹配质量，再填进去。像 Jina Reranker、M3E + BGE 组合在 C-MTEB 上能涨 5~8 个点，比静态融合强不少。

当大模型上下文窗口扩展到100万token时，哪些现有业务场景可能发生质变？

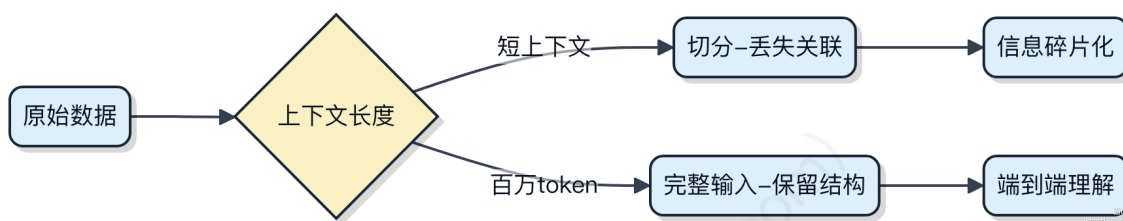
上下文窗口突破百万 token，最直接的影响是模型能“看见”和处理的信息长度远超以往。以前处理长文本得切片、摘要、丢信息，现在可以直接喂完整文档，很多场景的实现方式会彻底改变。

1) 金融领域的财报分析、合同审查会变得精准得多。比如分析一份上百页的跨国并购协议，模型能一次性理解所有条款的关联性，找出潜在风险点，而不用像过去那样分段处理导致上下文断裂。类似 **Kensho** 这类系统在做法律与金融文档分析时，准确率和效率会跃升。

2) 代码工程的理解和维护进入新阶段。整个大型项目的源码可以作为上下文输入，让模型真正理解模块间的依赖关系。像 **GitHub Copilot** 这种工具，不再只是函数级补全，而是能做跨文件重构建议、架构优化提示，相当于有个全局视角的资深架构师辅助。

3) 科研文献综述和知识发现会发生质变。研究人员可以把几千篇论文全文丢给模型，让它自动梳理领域发展脉络、发现研究空白。这在生物医药、材料科学这类文献密集型领域尤其关键。

4) 客服和企业知识库的体验升级。传统 RAG 面对复杂问题经常答非所问，因为检索片段有限。百万上下文下，可以直接加载整个产品手册或历史工单，回答更完整准确。



当发现RAG系统召回结果与用户query意图不匹配时，有哪些可能的改进方向？

RAG 系统召回结果偏离用户意图，问题通常出在检索和生成两个环节的衔接上。先定位是“没找到”还是“找到了但没用好”。

1) 优化检索阶段的语义匹配能力

query 本身可能有歧义或表述不清晰，直接搜索容易翻车。可以用 query 改写来增强表达，比如用大模型把原始 query 扩展成多个相关问题，再并行检索。像 LangChain 提供了 `MultiQueryRetriever` 就是干这个的。另外，embedding 模型是否适配业务场景也很关键，通用模型如 `text-embedding-ada-002` 在垂直领域可能不如微调过的 BGE 或 m3e。

2) 提升文档切分的合理性

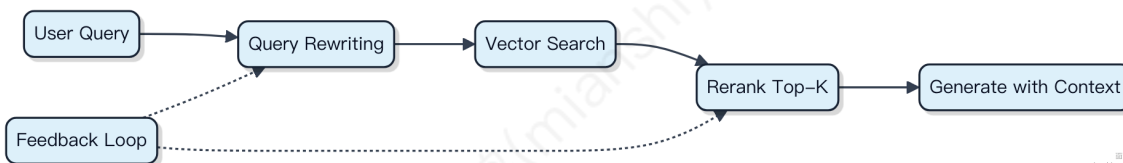
chunk 太大，关键信息被稀释；太小，上下文丢失。一般建议按语义切分，比如用 LangChain 的 `RecursiveCharacterTextSplitter` 结合 `sentence transformers` 做句子边界感知。chunk size 控制在 256~512 token 比较常见，具体得看内容密度。

3) 引入重排序 (Rerank) 机制

初检可能召回一堆相关度一般的文档，加一层 cross-encoder 重排能显著提升 TopK 质量。像 Cohere 的 reranker、bge-reranker 都可以直接调用，把最相关的几个往前顶，让 LLM 看到更准的信息。

4) 调整生成阶段的提示工程

有时候检索是对的，但 prompt 写得太松，模型自己脑补。要明确约束“基于以下上下文回答”，并在输入里只放高相关度的 chunk。也可以做 query 路由，判断是否走检索流程，避免无关 query 强行查库。



线上可以接反馈闭环，比如用户点赞/点踩驱动 embedding 或 reranker 微调，持续迭代。

使用LangChain实现RAG系统时，如何处理PDF文档中的表格数据召回问题？

PDF里的表格数据在RAG里是个硬骨头，因为传统文本切片会把表格拆得支离破碎，导致召回时压根不完整。LangChain本身不直接解析表格，得靠外部工具先把表格结构化。

1) 先用 `PyMuPDF` 或 `pdfplumber` 提取PDF中的表格，保持行列结构，转成HTML或Markdown格式存储。比如一个财务报表，直接按文本切片会丢失表头和对应关系，而用 `pdfplumber` 能还原成真正的二维结构。

2) 把结构化后的表格内容作为独立文档单元 (Document) 传给LangChain，设置特殊元数据标记，比如 `{"source_type": "table", "page": 5}`，方便后续检索时识别来源。

3) 向量化时，表格整体作为一个chunk嵌入，避免行级拆分。可以配合Chroma或FAISS做混合检索，在查询时判断是否涉及“金额”、“统计”等关键词，动态提升表格类chunk的召回权重。

4) 如果表格特别大，考虑用“表头 + 行”组合方式生成多个子chunk，但每条都保留完整表头信息，确保语义完整。

```
doc = Document(  
    page_content=markdown_table, # 结构化后的表格  
    metadata={"type": "table", "page": pagenum}  
)
```

整个流程关键在于**提前结构化**，而不是指望向量模型自己理解破碎的表格文本。很多RAG效果差，其实是数据预处理没做好这一步。

什么是 RAG 中的 Rerank? 具体需要怎么做?

RAG 里的 Rerank 其实就是对检索阶段召回的多个文档片段，重新打分排序，挑出最相关的结果喂给大模型生成答案。

一般流程是先用向量数据库做相似度搜索，比如从知识库找出 top-5 的候选文档。但向量检索有个问题，它只看语义相似，不一定和用户问题真正匹配。这时候就需要 Rerank 模型再筛一遍。

Rerank 模型通常是交叉编码器（Cross-Encoder），比如 BGE-Reranker 或 Cohere Rerank API。它会把用户问题和每个召回的文档拼在一起输入模型，输出一个相关性分数。这个过程比向量检索慢，但精度高得多。

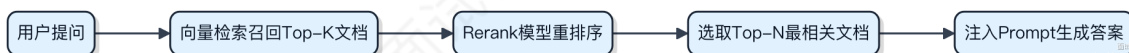
具体做法很简单，假设你有 5 个候选段落：

1) 把 query 和每个段落组合成一对文本 2) 送进 Rerank 模型得到相关性得分 3) 按得分从高到低排序，取前 2~3 个高质量段落输入 LLM

代码上类似这样：

```
# 伪代码示意  
pairs = [(query, doc) for doc in retrieved_docs]  
scores = reranker.predict(pairs)  
ranked_docs = [doc for _, doc in sorted(zip(scores, retrieved_docs), reverse=True)]
```

要不要加 Rerank 得看场景。如果你的系统能接受一定噪音，直接用向量检索也行。但要是追求准确率，尤其是客服、医疗这类容错低的场景，这一步的提升非常明显，基本能提 **10%~20%** 的最终回答准确率。



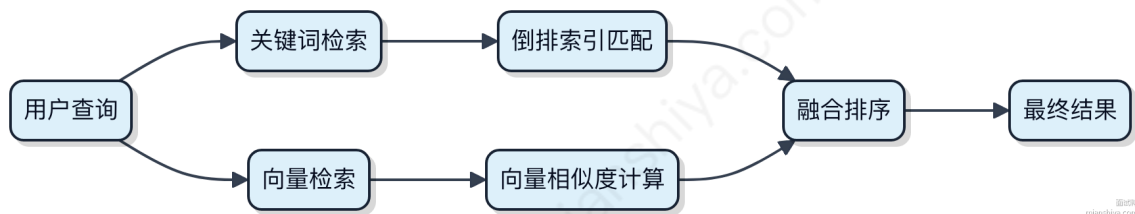
什么混合检索? 在基于大模型的应用开发中，混合检索主要解决什么问题?

混合检索不是简单把关键词和向量拼在一起。它要解决的是单一检索方式的短板，比如纯关键词匹配会因为字面不一致丢内容，纯向量检索又可能偏离精确条件。

1) 关键词检索基于倒排索引，查的是词频和位置，适合处理明确实体、过滤条件，像“2024年发布的手机”。但遇到语义相近表达，比如“续航强”和“电池耐用”，就容易漏。

2) 向量检索把文本映射到语义空间，靠相似度打分，能捕捉“高性能手机”和“游戏手机”这种隐含关联。问题是没法精准控制范围，比如指定价格区间或品牌，还可能把“苹果手机”和“水果苹果”搞混。

混合检索的关键是融合策略。常见做法是分别召回结果，再用加权、交叉排序（RRF）合并。比如 Elasticsearch 的 `query_then_fetch` 结合 `knn` 查询，或者用 RAGFlow、LlamaIndex 这类框架统一调度。



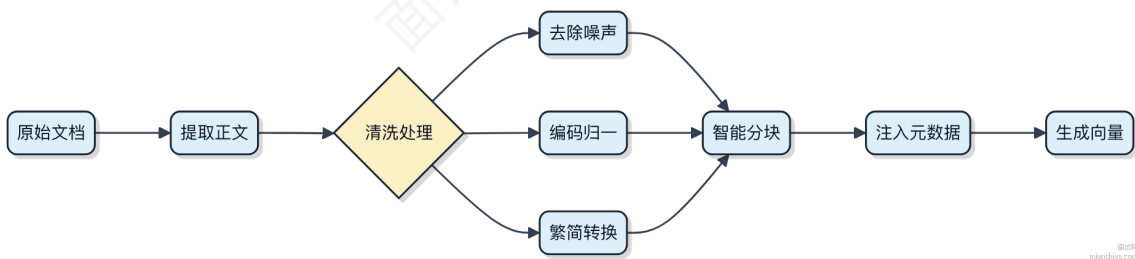
这招在问答系统、知识库检索里特别实用。你既要语义理解能力，又要支持字段过滤和精确匹配，光靠大模型生成不行，得靠混合检索把底座打好。

在 RAG 应用中为了优化检索精度，其中的数据清洗和预处理怎么做？

数据清洗和预处理在 RAG 中直接决定检索质量，很多效果差的问题其实都出在这一步。

- 1) 文本切分要合理，不能简单按固定长度硬切。比如用 LangChain 的 `RecursiveCharacterTextSplitter` 时，得保留段落上下文，避免把一句话从中间劈开。一般 `chunk size` 设成 512~~1024~~ token，`overlap` 保持 100~~200~~ token，让前后文能衔接。
- 2) 清理噪声是脏活累活。HTML 标签、广告文本、无意义的符号都得去掉。可以用 `BeautifulSoup` 或 `trafilatura` 提取正文，再通过正则过滤掉乱码和特殊字符。像维基百科这类数据源，还得剔除参考文献、编辑提示这些非主体内容。
- 3) 元数据注入很关键。给每个 chunk 加上来源 URL、标题、章节名等信息，召回时能结合上下文重排序。比如同一个文档的不同片段，可以通过 `source` 字段做去重或聚合。
- 4) 编码统一别忽视。所有文本转成 UTF-8，避免后续向量化时报错。中文场景下还要考虑繁简归一化，像“数据库”和“資料庫”最好映射到同一语义空间。
- 5) 可选做实体增强。用 `spaCy` 或 `HanLP` 抽关键词、人名、地名，附加到 chunk metadata 里，在检索时支持基于实体的扩展查询。

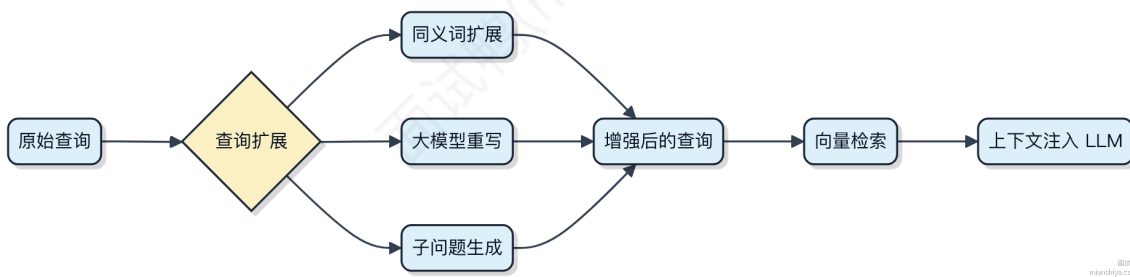
整个流程可以串成 pipeline，用 Apache Beam 或 Airflow 调度，确保数据进 Embedding 模型前是干净、结构化、带上下文的。



什么查询扩展？为什么在 RAG 应用中需要查询扩展？

查询扩展本质是把用户那句简单的原始问题，变形成一个或多个更完整、信息更丰富的查询语句。你别小看这一步，它直接决定了后续检索能不能捞出真正相关的内容。

- 1) 用户搜“苹果手机续航差”，其实他关心的是 iPhone 15 Pro 的电池表现和省电技巧。原始 query 太短，关键词稀疏，直接去向量库搜索容易匹配到无关的“苹果水果”或者泛泛的“手机耗电”。这时候就得靠查询扩展来补全意图。
- 2) 常见做法有几种。一种是同义词/近义词扩写，“续航”换成“电池寿命”“使用时间”；一种是基于大模型做**多跳推理**，让 LLM 把原始问题拆成几个子问题，比如“iPhone 15 Pro 满电能用多久”“哪些设置能延长待机”；还有一种是查询重写，把口语化表达转成更标准的检索语言。
- 3) 不做查询扩展的 RAG 系统，召回率通常很低。特别是面对缩写、俚语、模糊表达时，向量检索压根不经过语义深化，直接就去算相似度，结果就是 top-k 文档里没有命中关键段落。像 LangChain、LlamaIndex 都内置了 query transformation 模块，就是为了解决这个断层。

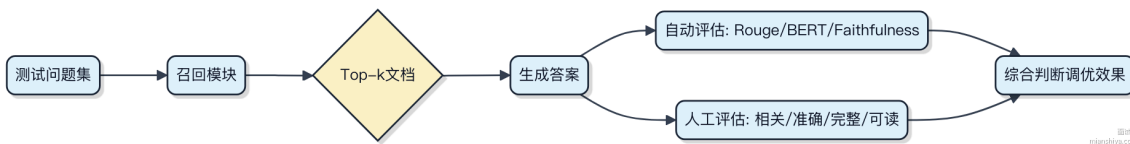


这步看着是脏活累活，但它是连接“人话”和“机器可检索语义”的关键桥梁。搞不定查询扩展，RAG 效果就很难稳定。

如何进行 RAG 调优后的效果评估？请给出真实应用场景中采用的效果评估标准与方法

RAG 系统调优后不能只看生成结果“好不好”，得用量化指标和真实场景反馈来判断是否真的提升了效果。重点是结合**自动评估指标和人工评估**，覆盖相关性、准确性和可用性。

- 1) 先看召回阶段，用传统信息检索指标。比如在企业知识库问答场景中，我们拿 1000 条历史用户问题做测试集，计算 top-5 文档的 **Recall@5** 和 **MRR**。如果 Recall@5 从 0.6 提升到 0.78，说明检索更准了，这是硬指标。
- 2) 再看整个 RAG 流程的输出质量。可以用 **Rouge-L** 衡量生成答案与标准答案的文本相似度，但别太依赖它。更关键的是 **BERTScore** 或 **BARTScore**，它们能更好捕捉语义匹配程度。比如在客服机器人中，我们发现 Rouge-L 变化不大，但 BERTScore 提高了 12%，实际回答更贴切了。
- 3) 引入事实一致性检测。用 **Faithfulness** 指标判断生成内容是否基于检索到的文档。比如模型说“报销周期是7天”，但文档写的是“5个工作日”，这就是幻觉。可以用专门打分模型如 RAGAS 来量化。
- 4) 最终绕不开人工评估。设计评分卡，让业务人员从**相关性、准确性、完整性、可读性**四个维度打分。比如在金融研报场景，分析师对调优前后的回答盲评，发现新版本在“数据来源清晰”这一项得分提升明显。



真实项目里，像用 LangChain + Pinecone 做法律咨询系统时，就是靠这套组合拳验证调优有效，上线后一次解决率提高了 23%。

在 RAG 中，你如何选择 Embedding Model 嵌入模型，需要考虑哪些因素？

选嵌入模型其实是个权衡过程，关键得看你的数据、场景和系统约束。

首先语言要对齐。如果你的业务全是中文，直接上 multilingual 或专门训过的中文模型，比如 **bge-base-zh** 这类，英文为主的模型在中文语义上容易翻车。反过来如果是多语言混合，可以考虑 mxbai、bge-m3 或 sentence-transformers 的多语言系列。

其次看向量质量与维度。高质量模型输出的向量更稠密，召回准确率高，但维度也高，比如 768 维会比 384 维占更多内存和计算资源。你要评估检索精度和性能之间的平衡。一般小模型适合轻量应用，大模型用在对相关性要求高的场景，像客服问答、精准推荐。

上下文长度也得留意。有的模型最大只支持 512 token，处理长文档时会被截断，语义就残了。如果做法律、论文这类长文本 RAG，得选支持 8k 甚至 32k 的模型，比如 bge-large 或者 jina-embeddings-v2。

推理成本不能忽视。开源模型自己部署可控，但要搞 GPU 推理服务，延迟和吞吐得压得住。如果不想管 infra，可以直接调用 API，比如阿里云的通义embedding、OpenAI 的 text-embedding-ada-002，省事但长期调用量大会贵。

最后还得和检索器匹配。比如你用 FAISS 做近似搜索，低维向量+余弦相似度效果好；要是用 Elasticsearch 向量库，可能就得适配它的距离函数。

一句话：模型不是越大越好，而是要跟数据、系统、成本三匹配。

在 RAG 中，索引流程中的文档解析你们怎么做的？

文档解析其实是 RAG 系统里特别关键的一步，搞不定这环，后面 embedding 再准也没用。我们一般不会直接扔原始文件给模型，得先把 PDF、Word、HTML 这些格式拆成干净的文本块，还得保留必要的结构信息。

- 1) 先用通用解析工具做预处理。比如 PDF 用 PyMuPDF 或 pdfplumber，能精准提取文字和分页；Office 文档用 Apache Tika，它支持格式多，省心得多。HTML 就用 BeautifulSoup 去掉脚本和广告这类噪音。
- 2) 接着是分块策略。不能简单按段落切，否则会把上下文割裂。我们会结合语义边界，比如标题层级变化时强制分块，同时控制每块在 300~500 token 左右。像 LlamaIndex 或 LangChain 提供的 RecursiveCharacterTextSplitter 就挺好用，能优先按 \n\n、句号这些符号切分。
- 3) 结构化信息保留也很重要。比如从企业文档中抽章节标题，打上 section=3.1 这样的元数据，后续检索时能结合标题过滤，召回准确率能提 20% 以上。

代码上基本长这样：

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=400,
    chunk_overlap=50,
    separators=["\n\n", "\n", "。", ". "]
)
chunks = splitter.split_text(text)
```

整个过程其实就是在“保语义”和“控长度”之间找平衡，太碎了影响上下文理解，太大了又容易混入无关内容。

你都了解哪些向量数据库？如何选型？

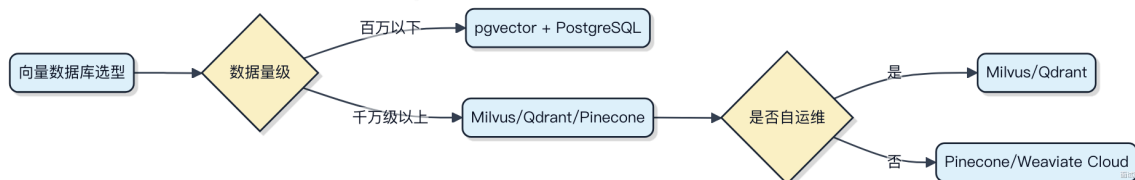
向量数据库这几年火起来，主要是为了解决高维向量的相似性搜索问题，尤其在大模型、推荐系统这些场景里用得越来越多。

常见的几个比如 **Milvus**，设计上就冲着大规模向量检索去的，支持 GPU 加速，能跟 Spark、Flink 这些大数据工具打通，适合数据量动不动就上亿的场景。另一个是 **Pinecone**，完全托管，你压根不用管运维，适合想快速上线又不想操心底层部署的团队。还有 **Weaviate**，它把向量和图结构结合得不错，如果你的数据本身有复杂关联关系，比如知识图谱，它会更合适。**Qdrant** 是 Rust 写的，性能强，内存控制也精细，API 设计得挺干净，最近不少新项目在用。

选型的时候先看数据规模和查询延迟要求。小项目几十万向量，其实 PostgreSQL 加上 pgvector 扩展就够了，没必要上重型方案。中等以上规模，就得考虑分布式能力，Milvus 和 Qdrant 都支持分片和副本，能扛住高并发。然后看生态集成，比如你已经在用 Kubernetes，那 Qdrant 的云原生支持就更顺滑。如果追求零运维，Pinecone 这类 SaaS 产品省心。

别忘了持久化和更新频率。有些库对动态删除支持不好，比如早期的 FAISS 就是纯内存的，现在虽然有变种但还是得留意。实时插入频繁的场景，得确认 WAL 或类似机制是否健全。

最后一点，向量模型一旦换过，老数据就得重新索引，迁移成本不小，一开始就把 schema 和版本管理设计好，后面少踩坑。



向量数据库原理是什么？请简述下它的原理

向量数据库本质是为高维向量数据设计的存储与检索系统，核心目标是在亿级向量中快速找到和查询向量最相似的那些。

传统数据库用 B+ 树做精确匹配或范围查询，但向量相似性搜索要的是“近似”，比如余弦相似度、欧氏距离。如果暴力遍历所有向量算距离，1 亿个向量每个 128 维，一次查询就得几十毫秒甚至上百毫秒，根本扛不住。所以关键在于 **近似最近邻搜索 (ANN) 算法**，在可接受精度损失下把查询从 $O(n)$ 降到 $O(\log n)$ 甚至常数级。

常见实现方式有几种：

- 1) 基于 **局部敏感哈希 (LSH)**，把相近向量以更高概率映射到同一个桶里，适合高维稀疏场景，但召回率波动大；
- 2) 基于 **树结构** 如随机投影树，划分空间，但高维下效果退化明显；
- 3) 最主流的是 **图-based 方法**，比如 HNSW (Hierarchical Navigable Small World)，构建多层导航图，顶层稀疏快速定位方向，底层精细搜索，能在毫秒内完成百万级查询，Milvus、Pinecone 都在用。

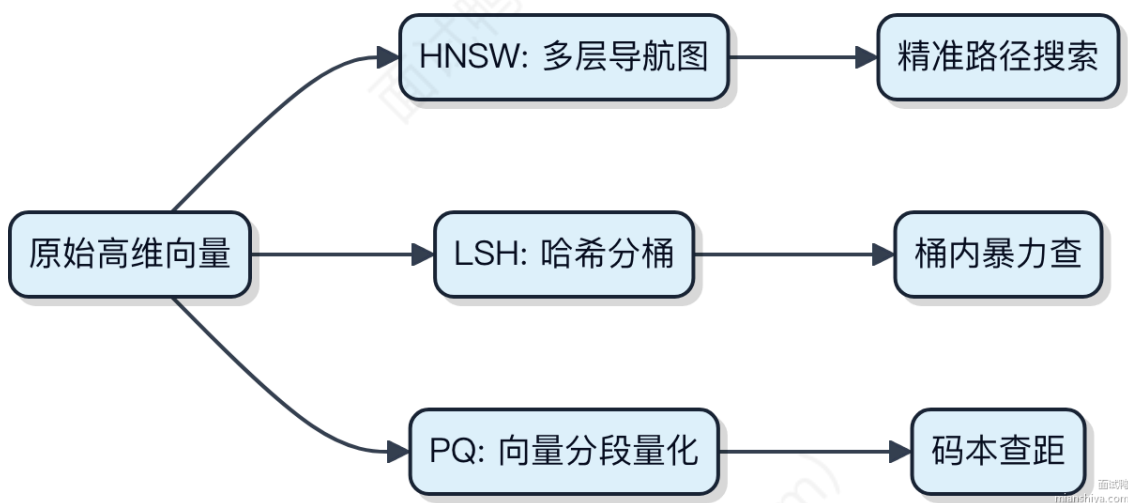
另外，向量数据库通常会结合标量索引做混合查询，比如“找和这个图片相似且上传时间在最近7天的”。这就需要支持结构化字段过滤 + 向量检索的联合优化。



向量数据库中的 HNSW、LSH、PQ 分别是什么意思？

HNSW、LSH、PQ 是向量数据库里常用的 **近似最近邻搜索**（ANN）技术，用来在高维空间快速找相似向量，毕竟暴力遍历 $O(n)$ 在亿级数据上根本扛不住。

- 1) **HNSW**（Hierarchical Navigable Small World）本质是个多层图结构。每一层都是个近邻图，高层稀疏用来“跳”得快，底层密集做精细搜索。查询时从顶层开始往下走，像导航一样逐步逼近目标点。Milvus、Weaviate 都用它，召回率高，延迟也稳，但建索引内存开销大。
- 2) **LSH**（Locality-Sensitive Hashing）思路更粗暴：把相似的向量通过特定哈希函数尽可能甩到同一个桶里。查的时候只搜对应桶，大幅缩小范围。优点是实现简单、存储省，但召回率不如 HNSW，尤其分布不均时容易漏。早期 Annoy 用过类似思想。
- 3) **PQ**（Product Quantization）干的是降维压缩的脏活。它把高维向量切成几段，每段独立聚类，用聚类中心代替原始向量分量，存个码本就行。原来 128 维 float 能压到 16 字节内，检索时用对称距离近似算相似度。Faiss 就拿它和 IVF 搭配，扛住了十亿级向量库。



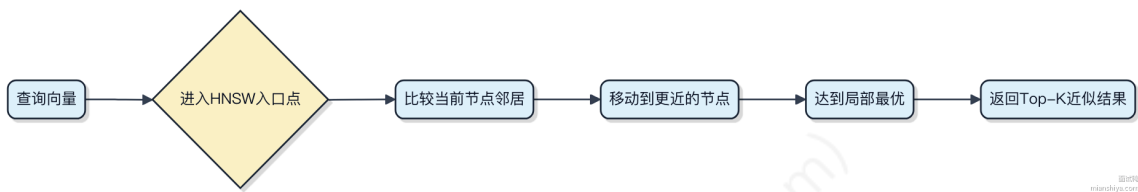
这仨经常混着用，比如 Faiss 里 IVFPQ 就是先用聚类定位候选集，再 PQ 压缩算距。选哪个看场景：要精度选 HNSW，要省资源可以试 LSH + PQ 组合。

向量数据库中的 ANN 是什么？为什么需要用它？

近似最近邻（ANN）是向量数据库里用来加速相似性搜索的核心技术。原始的暴力搜索要算查询向量和所有向量的距离，数据量一大，比如上亿条，根本没法实时响应。ANN 的思路就是牺牲一点点精度，换来几倍甚至上百倍的速度提升。

它通过构建特殊的索引结构来缩小搜索范围。常见的方法有：

- 1) 基于图的 HNSW，把向量连成近邻图，从一个入口点开始“爬山”，逐步逼近最近的邻居。Faiss 和 Milvus 都支持这种，效果好，吞吐高。
- 2) 基于聚类的 IVF，先把向量聚成若干簇，搜索时只查离查询点最近的几个簇，避免全量扫描。适合大数据集，配合 PQ 编码还能大幅降内存。
- 3) 基于 LSH 的哈希映射，把相似向量尽量散列到同一个桶里，但召回率在高维下容易掉得厉害，现在用得少了。



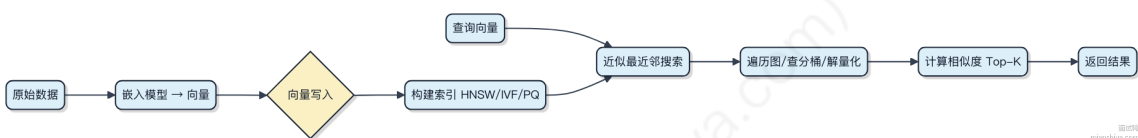
为什么非要用它？因为纯 brute-force 在千万级以上向量场景下，延迟动辄几百毫秒甚至秒级，压根不经过。而 ANN 能把响应控制在几毫秒内，才能支撑推荐、图文搜、语义匹配这些实时业务。像推荐系统里用户一刷就触发一次向量化检索，不用 ANN 根本扛不住。

向量数据库的工作流程有哪些？请简述下

向量数据库不是简单存个数组就完事了，它得解决“相似性检索”这个核心问题。整个流程从数据进来开始，到最后返回结果，每一步都围绕**高效近似最近邻搜索**展开。

- 1) 数据预处理阶段，原始数据比如文本、图片先过嵌入模型，转成高维向量。这步通常在外部做，比如用 Sentence-BERT 或 CLIP 模型生成向量，维度常见 384 到 1024。
- 2) 向量写入时，数据库会做索引构建。直接暴力计算所有距离肯定搞不定，所以要用专门的索引结构，比如 HNSW 构建图索引，IVF 做聚类分组，或者 PQ 做量化压缩。这些技术能让查询时跳过大部分不相关的向量。
- 3) 查询阶段，输入一个目标向量，系统先定位可能相近的候选集。比如 HNSW 从入口点出发沿图遍历，IVF 先算落在哪个聚类中心附近，再只搜那个分区里的向量。最后对候选集做精确距离计算，返回 Top-K 最相似的结果。

整个过程的关键在于**平衡精度和性能**。索引构建会影响写入速度和内存占用，而搜索策略决定延迟和召回率。典型产品像 Milvus、Pinecone、Weaviate 都是基于这类流程设计的。



PEFT 是什么？为什么需要 PEFT？

PEFT 全称是 Parameter-Efficient Fine-Tuning，直白点说就是“只动一小部分参数来微调大模型”。现在大模型动不动就上百亿、上千亿参数，比如 Llama、ChatGLM 这种，直接全量微调成本太高了，显存、算力、时间都吃不消。

其实我们发现，微调的时候没必要更新所有参数。PEFT 的思路是冻结原模型绝大部分权重，只训练少量额外引入或选中的参数，就能达到接近全量微调的效果。这样显存能省下 70% 以上，训练速度也快很多，普通几张卡甚至单卡也能搞。

常见的 PEFT 方法里，**LoRA** 是最火的。它在原始层旁边加个低秩矩阵做增量，训练时只更新这个小矩阵。比如一个 7B 模型，用 LoRA 可能只改 0.1% 的参数，效果却不差。HuggingFace 的 `peft` 库已经支持得非常好，配合 `transformers` 能轻松上手。

代码长这样：

```

from peft import LoraConfig, get_peft_model

lora_config = LoraConfig(

```

```

r=8,
lora_alpha=16,
target_modules=["q_proj", "v_proj"],
lora_dropout=0.1,
bias="none",
task_type="CAUSAL_LM"
)
model = get_peft_model(model, lora_config)

```

除了 LoRA，还有 Adapter、Prefix-tuning 等方法，但 LoRA 因为无推理延迟、实现简单，成了主流选择。特别是资源有限又想定制大模型的场景，比如企业私有模型微调，PEFT 几乎是标配方案。



参数高效微调（PEFT）的核心思路是什么？列举 3 种典型方法

参数高效微调的核心思路是只微调模型中**极小一部分**参数，冻结绝大部分原始参数，从而在保持下游任务性能的同时，大幅降低计算和存储开销。

大模型动辄上百亿参数，全量微调需要几十甚至上百 GB 显存，普通团队根本搞不定。PEFT 的做法是“以小博大”，通过引入少量可训练参数来适配新任务，比如只调 0.1%~1% 的参数就能达到接近全量微调的效果。

1) LoRA (Low-Rank Adaptation)

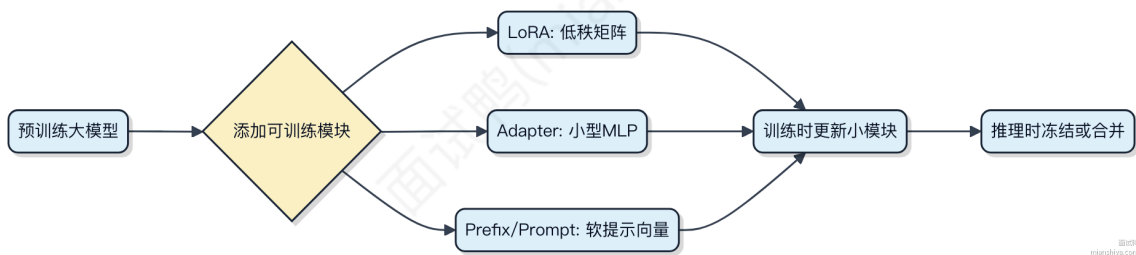
在原始权重旁注入低秩矩阵，训练时只更新这些小矩阵。推理时可以合并到原权重中，完全不增加推理延迟。HuggingFace Transformers 已深度集成，现已成为主流选择。

2) Adapter

在 Transformer 模块间插入小型前馈网络（通常两层 MLP），原模型冻结，只训练插入的模块。虽然效果好，会增加推理延迟，因为每次前向都要走额外网络。

3) Prompt Tuning / Prefix Tuning

通过可学习的“软提示”向量引导模型输出，本质是把任务建模成“填提示”的过程。Prefix Tuning 把这些向量拼接在每一层输入前，Prompt Tuning 只加在第一层。节省参数，但对初始化敏感，小模型上容易不稳定。



PEFT 和全量微调的区别？

PEFT 本质上是解决大模型微调成本太高的问题。全量微调要更新整个模型所有参数，显存和计算资源消耗巨大，比如一个 13B 的模型，光梯度存储就得上百 GB 显存。

而 PEFT 的思路是冻结原模型大部分参数，只训练一小部分新增或改造的模块。这样一来，显存占用能降到原来的 1/10 甚至更低，普通单卡也能跑。

1) 全量微调：每个参数都参与更新，效果理论上最好，但训练慢、成本高，需要完整的优化器状态和梯度保存。适合有充足算力且对性能极致要求的场景，比如公司级训练集群搞通用能力升级。

2) PEFT 方法：以 **LoRA** 为例，它在原始权重旁并行注入低秩矩阵，训练时只更新这些小矩阵。假设原始权重是 $W \in \mathbb{R}^{768 \times 768}$ ，LoRA 拆成两个小矩阵 $A \in \mathbb{R}^{768 \times r}$ 和 $B \in \mathbb{R}^{r \times 768}$ ，其中 r 通常设为 8 或 16。这样可训练参数数量从几十亿骤降到百万级。

```
// 伪代码示意 LoRA 注入
Matrix W = loadOriginalWeight(); // 冻结
Matrix A = randomInit(768, 8);   // 可训练
Matrix B = randomInit(8, 768);   // 可训练
Matrix loraOutput = input * (W + A * B); // W 不更新
```

常见变体还有 Adapter Tuning、Prefix Tuning 等，但 LoRA 因其不改变推理结构、兼容性好成为主流。HuggingFace 的 `peft` 库已经支持多种模式，配合 `transformers` 几行代码就能上。

效果上，多数任务中 LoRA 能达到全量微调 95% 以上的性能，但对强推理或复杂指令遵循任务可能略有差距。关键是省了太多资源，让 7B/13B 模型在消费级 GPU 上也能快速迭代。

在进行 Fine-Tuning 时，如何选择适合的预训练模型？

这个问题其实是在问怎么给特定任务挑一个合适的预训练模型来微调，关键不是模型越大越好，而是看匹配度和资源约束。

- 1) 先看任务类型对不对口。比如做中文文本生成或理解，那肯定优先选在中文语料上预训练过的模型，像 Qwen、ChatGLM 这类原生支持中文的系列就比纯英文的 LLaMA 更合适。语言不匹配的话，再大的参数量也搞不定语义理解。
- 2) 再看模型规模和你的算力能不能对上。7B 参数的模型用单卡 A100 能跑，但如果是 70B，就得考虑模型并行或者直接上云了。一般情况下，中小公司做垂直场景微调，7B 左右的模型性价比最高，显存压得下来，训练时间也扛得住。
- 3) 还要看预训练数据的时间范围和领域。比如你要做金融研报生成，最好选那些在财经文本上持续预训练过的模型，而不是通用语料训完就停的。有些开源模型会标明训练数据来源和时间，这点要仔细查文档。
- 4) 最后看微调生态支不支持。像 Hugging Face 上有大量基于 LLaMA 系列的 LoRA 微调工具链，如果你选了一个冷门模型，可能连适配的 Trainer 都没有，脏活累活都得自己写。

代码上其实就是加载不同 checkpoint 的区别：

```
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("qwen-7b")
model = AutoModelForCausalLM.from_pretrained("qwen-7b")
```

换模型时路径一改就行，但背后的数据和架构差异决定了效果上限。

微调中常用的优化器有哪些？

微调大模型时选优化器，关键得平衡收敛速度和训练稳定性。常见的几种其实都源自梯度下降的改进思路。

- 1) SGD with Momentum 算是老前辈了，加个动量项能让参数更新不那么容易在平坦区域震荡。但学习率敏感，调不好容易不收敛，现在大模型微调用得少了。

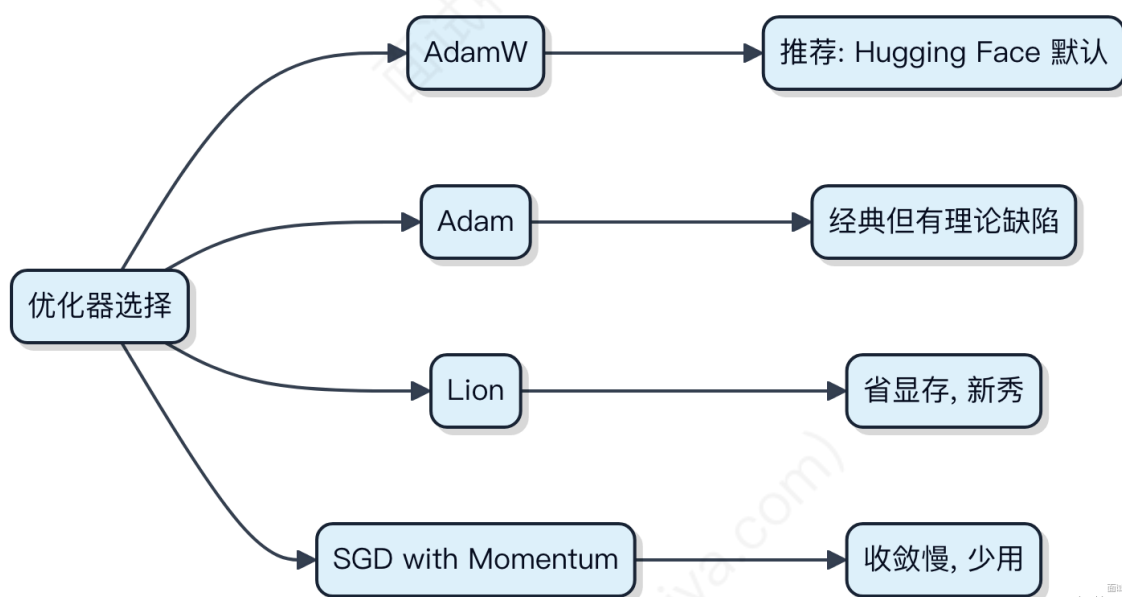
2) Adam 最常见，几乎成了默认选项。它自适应调整每个参数的学习率，对稀疏梯度特别友好。BERT、RoBERTa 这些预训练模型做下游任务微调时，基本都拿 Adam 打底。不过内存开销大，因为要存每个参数的一阶二阶梯度。

```
// PyTorch 伪代码示意，实际不用 Java 写训练
optimizer = torch.optim.Adam(model.parameters(), lr=2e-5)
```

3) AdamW 是 Adam 的升级版，把权重衰减从梯度更新里拆出来单独算，解决了 Adam 在正则化上的理论缺陷。Hugging Face Transformers 里默认推荐的就是 AdamW，像微调 BART 或 T5 都这么配。

4) Lion 是较新的选择，Google 提出来的，只用一阶动量，内存比 Adam 少一半，而且在某些 NLP 任务上效果更好。但它对学习率更敏感，需要更精细地调参。

总的来说，**AdamW** 因为稳定性和效果兼备，成了当前主流选择。小规模实验可以试试 Lion 看能不能提速，老项目可能还在用 Adam。SGD 基本只出现在特定场景，比如某些视觉模型微调还会用。



如何判断微调效果是否达到预期？

微调效果能不能扛住业务，得靠数据说话，不能拍脑袋。

1) 先看基础指标，比如分类任务的准确率、F1 值，生成任务的 BLEU、ROUGE。这些是硬指标，训练集和验证集上的提升得稳定，一般要求验证集指标比基线模型高 **5% 以上** 才算有显著收益。

2) 光看数字不够，得做人工评估。抽样一批模型输出，让业务方或标注人员打分，重点关注生成内容是否符合语义、逻辑通顺、不胡说八道。像用 Qwen 或 Llama 微调后，常出现“套话连篇但答非所问”的问题，这种机器指标发现不了。

3) 上线前必须走 A/B 测试。把老模型和新模型同时跑在线上，按流量切分，对比点击率、用户停留时长、转化率这些业务指标。比如在客服场景用微调模型，目标是降低人工转接率，那这个值下降 **10% 以上** 才叫真有效。

4) 还得看泛化性。拿一些边缘 case 或分布外数据测试，比如新增的用户 query 类型，模型能不能兜住。如果一遇到没见过的句式就崩，说明过拟合了，得回炉加数据增强或者正则。

代码层面可以写个简单的评估脚本：

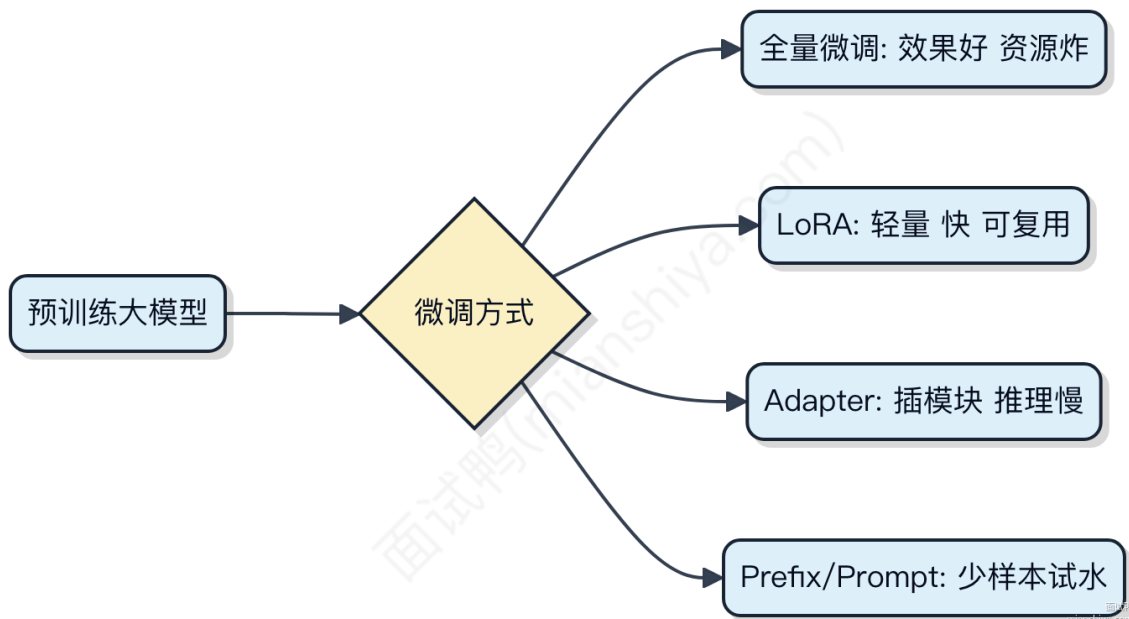

```
from sklearn.metrics import f1_score
f1 = f1_score(y_true, y_pred, average='weighted')
print(f"微调后F1: {f1:.3f}")
```

最后记住，**过拟合**是微调最大的坑，训完训练集接近 100% 准确率，验证集却卡在 70%，基本就是背答案了。数据质量差、样本太少、学习率太高都可能引发这问题。

介绍几种常见的微调策略的优缺点

微调大模型不是上来就全量参数一把梭，得看资源和场景做选择。

- 1) **全量微调**把所有参数放开训练，效果最好，像 Llama 系列在特定任务上刷榜基本都靠它。但代价也大，一张 A100 训个 7B 模型就得跑好几天，显存轻松干到 80G 以上，小团队根本搞不定。
- 2) **LoRA（低秩适配）** 是现在最火的轻量方案。它冻结原模型，在输入层加两个低秩矩阵做增量，训练时只更新这两块。比如 HuggingFace 上很多基于 Qwen、ChatGLM 的衍生模型都是用 LoRA 微出来的。显存能压到 20G 内，训练快，还能多任务共用一个底模型。缺点是超参敏感，r 秩选不好容易欠拟合。
- 3) **Adapter Tuning** 在 Transformer 层中间插入小型神经网络模块，训练时只动这些小模块。结构改动比 LoRA 大，推理有延迟，但现在基本被 LoRA 吊打，除非你对模块化设计有强需求。
- 4) **Prefix Tuning** 和 **Prompt Tuning** 都是往输入前面塞可学习的向量。前者改 attention key/value，后者直接拼接 embedding。适合少样本场景，但长序列下效果不稳定，一般只用于探索性实验。



一般来说，**LoRA** 是平衡效果和成本的首选，真要拼上限再考虑全量。

向量数据库中，常见的向量搜索方法：余弦相似度、欧几里得距离和曼哈顿距离分别是什么？有什么区别？

向量搜索的本质是衡量两个向量之间的“距离”或“方向相似性”，不同度量方式适用于不同场景。

余弦相似度关注的是向量间的夹角，不关心长度。它把向量投影到单位球面上，算的是它们方向的一致性。值在 -1 到 1 之间，越接近 1 表示方向越一致。适合文本、图像 embedding 这类语义匹配场景，比如用 Milvus 或 Faiss 做相似句子检索时，默认往往就是余弦相似。

欧几里得距离是两点间的直线距离，公式就是 $\sqrt{\sum (x_i - y_i)^2}$ 。它对向量的绝对位置敏感，适合特征空间中坐标准确反映实际差异的情况，比如地理坐标聚类。但高维下容易“维度诅咒”，距离都拉得很开，区分度下降。

曼哈顿距离是各维度绝对差之和， $\sum |x_i - y_i|$ ，也叫 L1 距离。它沿着坐标轴走“网格线”，对稀疏数据更鲁棒，计算也轻。在某些稀疏 embedding 或特征工程中会用到，不过在主流向量化检索系统中用得少。

- 1) 余弦相似度看方向，适合语义匹配
- 2) 欧几里得距离看直线远近，适合低维稠密空间
- 3) 曼哈顿距离看网格步数，抗噪强但应用窄

选择哪种，得看数据分布和业务目标。比如推荐系统里用户向量归一化了，用余弦最稳；没归一化且维度不高，可以试试欧氏。



在 RAG 应用的过程中，关于提示工程的设计有什么心得和技巧吗？

提示工程在 RAG（检索增强生成）里不是简单拼接问题和文档，而是决定模型能不能把检索到的内容用对、用好的关键。设计得好，大模型能像专家一样精准回答；设计得不好，结果可能张冠李戴。

- 1) 上下文组织要有结构
别把一堆段落堆给模型。要把检索到的片段按相关性排序，加上来源标记，比如用 `【doc1】` 开头标注。这样模型更容易分辨信息出处，减少混淆。
- 2) 指令要明确约束输出行为
直接告诉模型“只基于提供的文档回答，不知道就说不知道”。避免它自由发挥。可以加一句：“如果信息不完整，请拒绝回答”，防止幻觉。

3) 利用模板提升稳定性

预定义提示模板，固定角色、任务和格式。例如：

你是一个金融客服助手，请根据以下材料回答用户问题。
回答要求：简洁准确，不超过 50 字，引用文档编号。

【doc1】中国2023年GDP增速为5.2%...

【doc2】美联储2023年加息三次...

问题：2023年中国GDP增速是多少？

回答：2023年中国GDP增速为5.2% 【doc1】

4) 动态截断控制 token 长度

长文档容易撑爆上下文窗口。一般用 BERTScore 或最大边际相关性（MMR）做去重和精简，保留最相关的句子。HuggingFace 的 sentence-transformers 可以快速实现这一点。

5) 加入推理引导提升逻辑性

对于复杂问题，可以在提示中引导分步思考。比如“先判断问题类型，再查找对应数据，最后组织语言”。

实际项目中，LangChain 和 LlamaIndex 都提供了提示模板管理功能，支持动态注入检索结果，建议结合使用。持续 A/B 测试不同模板对准确率的影响，才是长期优化的关键。

什么是 Advanced RAG?

Advanced RAG 是在基础 RAG（Retrieval-Augmented Generation）之上，通过优化检索和生成两个环节的协同效率，提升回答质量的一套方法论。它不是某个具体工具，而是一系列增强技术的组合。

1) 传统 RAG 通常直接用用户问题去向量库检索，但自然语言存在歧义或表述不清的问题，导致召回不准。Advanced RAG 会先做**查询重写**，比如用 LLM 把原始问题拆解成多个子问题，或者改写成更适合检索的关键词形式，像 DPR（Dense Passage Retrieval）这类模型就常被用来做语义对齐。

2) 在检索阶段，除了单纯找 top-k 最相似的文档块，还会引入**重排序**（Re-Ranking）。比如用 Cross-Encoder 对初步召回的结果做精细打分，把真正相关的排前面。这块可以结合 BM25 等稀疏检索做混合召回，提升覆盖率。

3) 生成环节也不再是“一把喂给 LLM”。有些方案会做**上下文压缩**，只提取出与问题最相关的句子片段，减少噪声和 token 消耗。还有像 Self-RAG 这样的思路，让模型自己判断是否需要检索、要不要采纳检索结果，增加推理可控性。

代码上其实变化不大，核心是流程增强：

```
# 伪代码：Advanced RAG 流程
rewritten_query = rewrite_prompt(user_query)
docs = vector_store.search(rewritten_query, k=10)
reranked_docs = cross_encoder_rerank(rewritten_query, docs)
context = compress_context(rewritten_query, reranked_docs)
answer = llm.generate(context, user_query)
```

整个链路更像是一个可调教的系统工程，典型应用有 LangChain + FAISS/BGE + Cohere Rerank 的组合，或者是用 LlamaIndex 做节点后处理。关键是要能根据业务场景动态调整各模块权重，而不是堆模块。

什么是 Modular RAG?

Modular RAG 不是某个固定框架，而是指一种将 RAG（检索增强生成）系统拆解成独立、可替换模块的设计思路。你完全可以把它看作 RAG 的“微服务化”——每个环节单独优化、灵活组合。

整个流程通常包括几个关键阶段：首先从外部知识库中检索相关文档，然后对检索结果做重排序（Rerank），最后把处理好的上下文喂给大模型生成答案。传统 RAG 把这些步骤耦合在一起，而 Modular RAG 强调每个部分都能独立迭代。

比如你在用 LangChain 或 LlamaIndex 时，可以自由替换不同的检索器（如从 BM25 切到向量检索）、换上 Cross-Encoder 做精排、再接入不同的 prompt 工程策略。这种结构下，一个团队可以专注优化检索质量，另一个团队负责提升生成流畅度，互不干扰。

1) 检索模块：支持多路召回，比如同时走关键词和语义向量 2) 重排序模块：用更精细的模型对 Top-K 结果打分，提升相关性 3) 上下文融合：对多个片段做去重、摘要或拼接，减少噪声 4) 生成控制：动态决定是否启用检索，避免“硬套答案”



这种方式特别适合复杂场景，像在企业知识库中查政策，可能需要结合时间过滤、权限判断等额外逻辑，直接塞进端到端 RAG 很难维护。拆成模块后，脏活累活交给对应组件，主链路反而更清晰。

Computer Use 是什么？说说它的原理

目前没有明确的技术标准或广泛认知的系统叫做 "Computer Use"。这个术语在主流计算机科学、软件工程或 IT 基础设施领域并不指代某个具体的技术、协议或架构。

如果你指的是“计算机的使用”（the use of computers），那涵盖范围非常广，包括操作系统调度、程序执行、内存管理、网络通信等基础行为。但作为一个专有名词，“Computer Use”并不是像 Redis、Kafka 或 JVM 这样的具体技术组件。

有可能是提问时表述有误，比如本意想问的是：

- **Compute Unit**：计算单元，常见于分布式计算框架（如 Spark 中的 executor）或硬件架构中。
- **Computer Vision**：计算机视觉，AI 的一个分支。
- **Computer Cluster**：计算机集群，用于高性能计算或服务扩容。
- 或者某个特定产品/项目中的内部命名，例如某系统的“ComputerUseService”类。

建议确认题目是否准确。如果是面试中遇到这个问题，可以先和面试官澄清概念，避免答偏。很多时候，这种模糊题是在考察你如何应对需求不明确的问题——这时候沟通能力和问题拆解能力比技术本身更重要。

与其纠结术语定义，不如展示分析思路：1) 询问上下文场景

2) 列举可能的相近概念进行排除

3) 基于合理假设给出解答路径

这样反而更容易拿到高分。

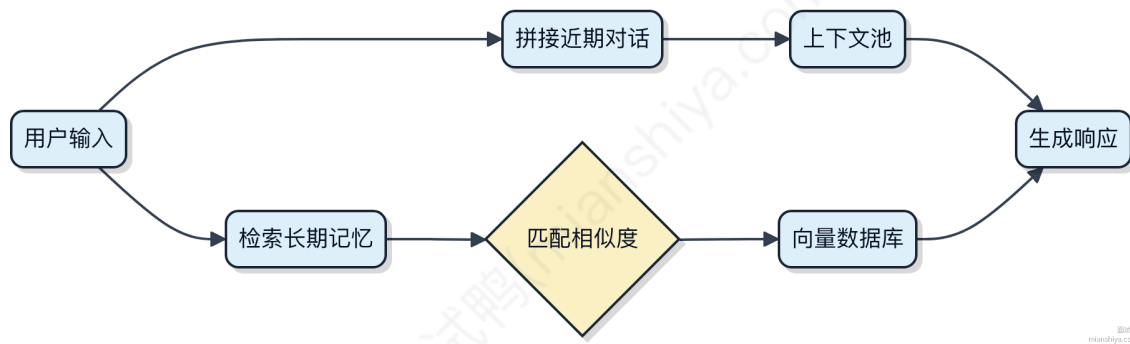
在大模型应用中，通常如何实现长短期记忆机制？

大模型本身没有内置的长期记忆，每次对话都是独立的。要实现长短期记忆，得靠外部手段和架构设计来补足。

短期记忆一般通过上下文窗口管理。模型能处理的 token 数有限，比如 32K，把最近的对话历史拼接进去就行。关键是怎么裁剪，别让上下文爆炸。常见的做法是滑动窗口、摘要压缩，或者用向量检索挑出最相关的几句历史，这样既

能保留关键信息，又不会超出长度限制。

长期记忆就得依赖外部存储了。用户的历史行为、偏好、重要事件这些数据，存到数据库里，比如用 **向量数据库** 存语义特征。每次新请求进来，先根据当前输入去检索最相关的历史片段，再把这些“记忆”注入 prompt。像 LlamaIndex 或 LangChain 这类框架，就提供了现成的记忆管理模块，支持会话级的状态保持。



状态同步也得考虑。如果应用是分布式的，会话状态不能只存在本地，得用 Redis 这种中间件做共享存储，保证每次请求都能拿到一致的历史。

说白了，模型只负责“想”，不负责“记”。所有记忆相关的脏活累活，都是外围系统在扛。

Copilot 模式和 Agent 模式的区别是什么？

Copilot 模式和 Agent 模式代表了 AI 编程辅助的两种不同层级。

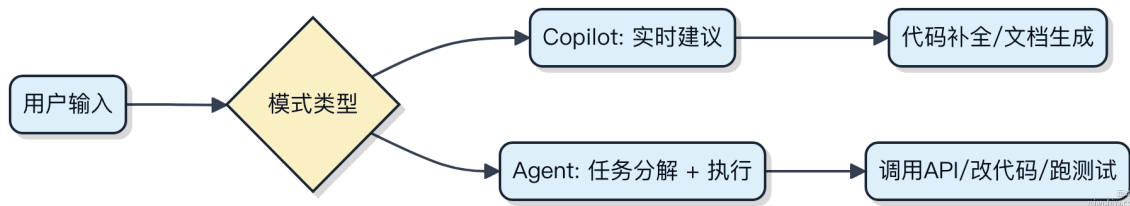
Copilot 更像是一个实时建议者，你在写代码时它在旁边提供建议。你敲下一行，它预测下一行，比如在 IntelliJ 或 VS Code 里补全一段方法体。它的响应延迟要求高，一般在 200ms 内给出提示，决策链路短，不会自己发起动作。典型的场景是 GitHub Copilot 补全函数、生成单元测试。

Agent 模式则更进一步，它能**自主规划**和执行任务。给你提个需求，比如“新建一个 Spring Boot 项目，集成 MySQL 和 Redis”，它能拆解任务、创建文件、写配置、甚至跑通 CI。整个过程不需要你一步步确认，它会自己调用工具、检查结果、修正错误。背后依赖的是 LLM 的推理能力加上外部工具链，比如 LangChain 或 AutoGPT 框架支持的插件系统。

关键区别在于控制权。Copilot 等你触发，输出是建议；Agent 自己决定下一步做什么，输出是完成的任务。

举个例子：

- 你写 `// 查询用户订单`，Copilot 接着帮你写出 SQL 查询代码。
- 你说“把用户查询功能加上缓存”，Agent 会分析现有代码，修改 DAO 层，加 RedisTemplate 调用，更新配置文件，还可能加上降级逻辑。



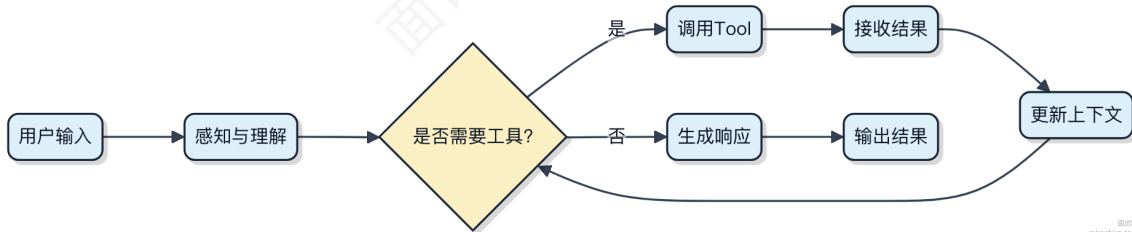
简单说，Copilot 是副驾驶，Agent 是自动驾驶。

Agent智能体的工作过程是怎样的？

Agent智能体不是简单的问答机器人，它更像一个能自己“思考”并采取行动的自动化代理。整个过程围绕目标驱动，通过感知、决策、执行和反馈闭环来完成任务。

- 1) 首先从环境或用户指令中获取输入，比如一段文字需求。接着进行规划，把大目标拆解成可执行的小步骤，这一步会用到提示工程里的思维链（Chain of Thought）或ReAct框架。
- 2) 进入工具调用阶段，如果当前任务需要外部信息，比如查天气、发邮件或者搜索数据库，Agent就会选择合适的工具（Tool）发起调用。这个过程通常由LLM判断该不该用、用哪个，并生成符合格式的请求参数。
- 3) 拿到工具返回结果后，再结合上下文做下一步推理，决定是继续调用其他工具还是直接生成最终回复。整个流程可以循环多次，直到达成目标。

代码层面一般基于LangChain或LlamaIndex这类框架搭建，它们提供了统一的Agent接口、Tool抽象和记忆管理机制。比如在LangChain里，你只需要定义好tools列表和llm实例，就能快速构建一个具备搜索+计算能力的Agent。



关键在于**自主性**和**动态规划**，不是预设流程，而是根据中间结果不断调整策略。但也要注意控制循环深度，避免陷入无限调用陷阱。

LLM Agent 如何进行动态 API 调用？

LLM Agent 做动态 API 调用，关键在于把自然语言意图转成结构化请求，中间靠 **工具抽象** 和 **运行时绑定** 撑起来。

Agent 不是直接拼 URL 调接口，而是通过定义好的工具描述（Tool Description），让模型知道“我能干啥”。比如你接入了飞书消息 API 或内部工单系统，就封装成 tool，带上 name、description、parameters，用 JSON Schema 描述输入格式。模型输出会明确说“我要用 send_message 这个工具，参数是 to=张三, content=上线提醒”，而不是自由发挥写一段话。

真正执行时，Agent 框架捕获这个调用意图，拿参数去实例化真实请求。这块一般有统一的 Tool Executor，注册所有可用工具，做参数校验、鉴权、重试，再发出去。像 LangChain、LlamaIndex 都是这么搞的，你自己写也建议拆成两层：一层对齐模型理解，一层处理 HTTP、序列化这些脏活累活。

代码上大概长这样：

```
record SendMsgTool(@NonNull String to, @NonNull String content) implements Tool {
    public void run() {
        // 实际调飞书 Webhook
        HttpClient.newCall(req -> req.url("https://open.feishu.cn/path")
            .post(json(to, content)));
    }
}
```

整个流程其实是“模型提议 + 系统执行”的分离设计。模型压根不经过网络层，只负责决策调哪个工具；真正的 API 调用由运行时安全落地，避免幻觉乱发请求。高频场景比如自动查数据库、触发 CI/CD、拉取监控指标，都是靠这套机制扛住的。

LangChain 的核心组件有哪些？

LangChain 的设计是为了解耦大模型应用的各个关键环节，让开发者能灵活组装。它不是单一工具，而是一套拼图。

1) Model I/O

负责与大模型交互，封装了输入输出逻辑。支持多种模型类型，比如 OpenAI、Anthropic、Hugging Face 等。你不用自己写 HTTP 请求，它帮你统一调用方式。

2) Prompts

提示词管理模块，包括提示词模板（PromptTemplate）和示例选择器（ExampleSelector）。你可以定义变量化的 prompt，比如把用户问题动态填入模板，还能做 few-shot 示例注入。

3) Chains

把多个步骤串起来执行。比如先调用模型生成摘要，再用另一个 prompt 做情感分析，整个流程封装成一个 chain。最常用的是 LLMChain，也可以自定义复杂链式逻辑。

4) Agents

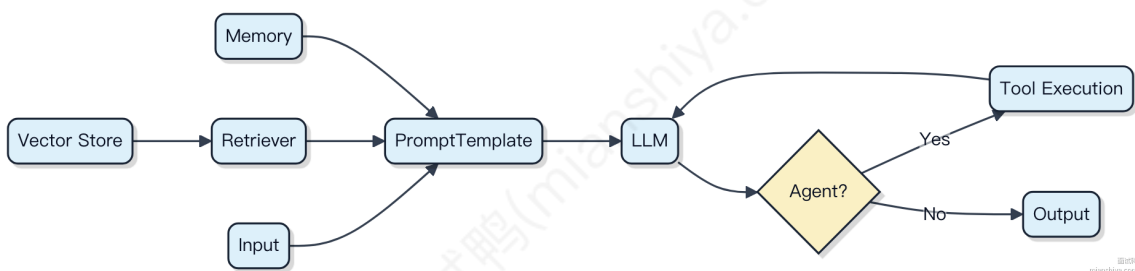
允许模型“决策”下一步动作。比如模型判断当前需要查天气，就会调用指定工具（Tool），执行完再继续推理。背后依赖 Tools 和 AgentExecutor，典型场景像 LangChain 结合 SerpAPI 做搜索代理。

5) Memory

维持对话状态，常见有 ConversationBufferMemory（保存全部历史）和 ConversationSummaryMemory（只存摘要）。适合聊天机器人这类需要上下文记忆的应用。

6) Retrieval

结合向量数据库做检索增强。比如用 FAISS 或 Chroma 存文档嵌入，查询时先检索相关片段，再喂给模型生成答案，有效缓解幻觉。



这些组件可以单独使用，也能组合出复杂应用，关键是按需选取，别一上来就全堆上去。

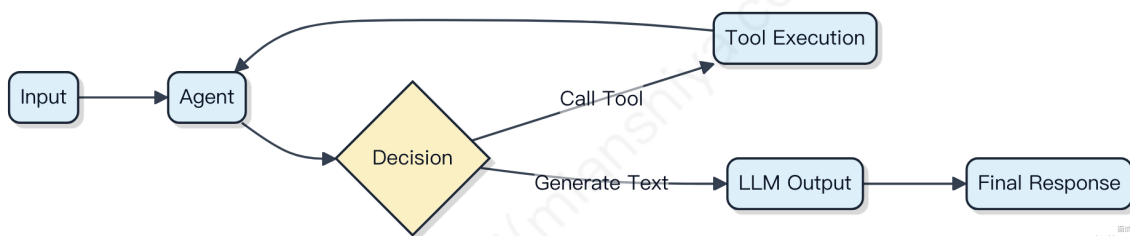
LangChain 核心架构是什么样的

LangChain 的设计其实围绕一个核心目标：让大模型能像搭积木一样灵活接入各种外部能力。它不追求把所有功能塞进引擎，而是通过标准化接口降低扩展成本。

链（Chain） 是整个架构的组织单元，你可以把多个处理步骤串成一条流水线。比如先用 PromptTemplate 生成文本，再交给 LLM 处理，最后调用 API 写入数据库，每个环节都是可替换的组件。

关键抽象有四个：1) Model I/O 封装了大模型的输入输出，支持 OpenAI、Anthropic、HuggingFace 等主流服务 2) Retriever 负责从外部获取数据，典型的是用 FAISS 或 Chroma 做向量检索 3) Tool 允许模型调用外部函数，比如搜索天气、执行 Python 代码 4) Agent 根据 prompt 决定调用哪些 Tool，实现动态决策流

实际项目里，你常会看到 Agent 结合 ReAct 模式工作。模型输出带特殊标记的文本，框架解析后触发工具调用，结果再拼回去继续推理。这种设计让逻辑闭环可以在一次会话中反复迭代。



和直接调 API 的最大区别是，LangChain 把“感知-决策-行动”循环变成了可配置的工作流。但要注意，链路越长，延迟叠加越明显，调试也越困难。简单任务直接用 LLMClient 反而更稳。

什么是 A2A 协议，它的核心架构及主要组件有哪些？

A2A (Application-to-Application) 协议不是某个具体的技术标准，而是描述系统间通过预定义接口直接通信的模式。这类交互通常在服务之间完成数据交换或功能调用，不依赖用户参与。

一般架构下，A2A 的核心是**去中心化或轻量网关控制的服务直连**。常见于微服务、B2B 集成或内部系统协同场景。比如订单系统自动触发库存扣减，就是典型的 A2A 调用。

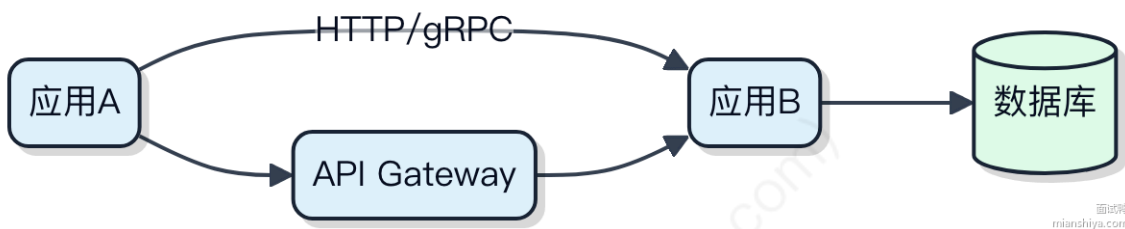
典型组件包括：

1) **服务提供方**：暴露 RESTful API 或 gRPC 接口，比如用 Spring Boot 实现的用户中心服务。2) **服务消费方**：主动发起请求的应用，例如交易系统调用风控服务做决策。3) **认证与授权模块**：常用 OAuth2 的 Client Credentials 模式，确保机器身份可信。API 密钥也常用于轻量场景。4) **API 网关 (可选)**：像 Kong 或 Apigee，负责限流、日志、路由，但高并发内网调用时常绕过网关直连以降低延迟。5) **服务注册与发现**：在动态环境中，Eureka 或 Nacos 帮助消费方找到可用实例。

代码上，一个简单的 A2A 调用可能长这样：

```
// 使用 WebClient 发起异步请求
webClient.get()
    .uri("http://user-service/users/123")
    .retrieve()
    .bodyToMono(User.class)
    .block();
```

注意点在于网络超时、重试策略和熔断机制。如果搞不定容错，雪崩效应会迅速扩散。因此，配合 Hystrix 或 Resilience4j 是常规操作。



A2A 协议有哪五大设计原则？

A2A（Application-to-Application）协议本质是服务间通信的契约设计，重点在于稳定、高效、可维护。它的五大设计原则其实围绕的是“解耦”和“可控”。

1) 接口版本化

接口一旦发布就不能随便改，否则上游调用方会炸。一般通过 URL 路径或 Header 携带版本信息，比如 `/v1/order` 和 `/v2/order` 并行存在，老接口逐步下线。

2) 明确的契约定义

用 OpenAPI（Swagger）、Protobuf 或 Thrift 明确定义输入输出结构，字段类型、是否必填、枚举值都要说清楚。前后端或跨团队协作时，契约先行，避免“我猜你返回的是个字符串”。

3) 错误码标准化

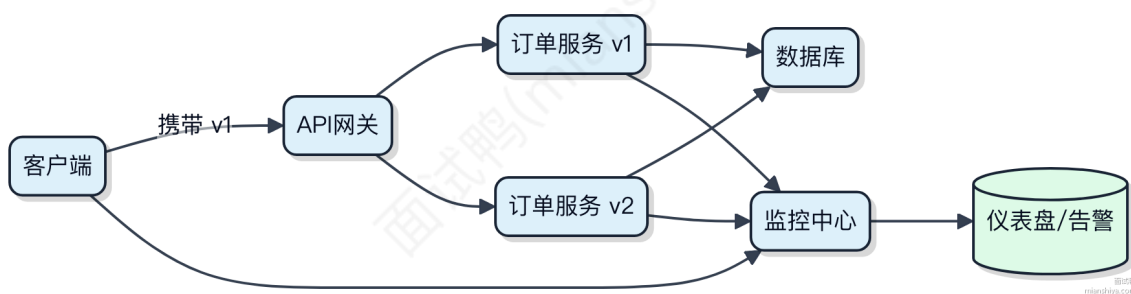
别随便返回 200 包着一个 `{code: 500}` 的体。HTTP 状态码要合理使用，业务错误码要统一规范，比如 `ORDER_NOT_FOUND=10001`，文档可查，日志可追踪。

4) 限流与熔断机制内建

系统扛不住的时候得自我保护。A2A 调用必须内置限流（如令牌桶）和熔断（如 Hystrix、Sentinel），防止雪崩。比如下单服务被查询服务拖垮，就得把非核心链路熔断掉。

5) 可观测性支持

每一次调用都能追踪，通过链路追踪系统（如 SkyWalking、Zipkin）记录请求路径，结合日志和监控指标（延迟、QPS），快速定位问题在哪一环。



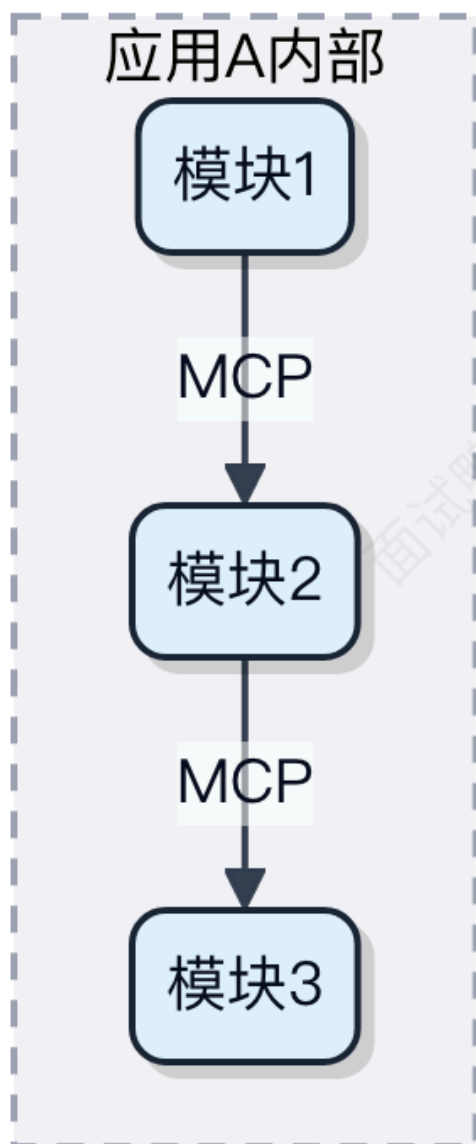
A2A 协议与 MCP 协议的关系是怎样的？

A2A 和 MCP 都是服务间通信的协议方案，但定位和层级不一样。

A2A 一般指 Application-to-Application，是业务应用之间的直接调用协议，比如通过 Dubbo、Spring Cloud RPC 或 gRPC 实现。这类调用压根不经过页面层，都是接口对接口，常见于微服务架构里订单系统调库存系统这种场景。

MCP 全称可能是 Module Communication Protocol，但在阿里系中间件语境下，它更多指一种模块级通信规范，通常运行在同一个应用内部的不同逻辑模块之间，或者是轻量级跨进程通信的封装。它的抽象层级比 A2A 低一些，更像是中间件内部组件交互的语言。

你可以把 A2A 看作城市之间的高速公路，两个独立城市（应用）之间跑整车货（请求体）。而 MCP 更像是一个工厂内部的传送带，不同车间（模块）之间传递零件，节奏更紧凑，协议更定制化。



所以两者不是替代关系，而是分层协作。A2A 解决的是系统边界的调用问题，MCP 处理的是内部模块间的协同，脏活累活分工明确。实际在 SOFAShark 或类似架构中，你会同时看到这两种协议在跑。

[提示词优化考虑哪些维度？你们提示词模板有哪些字段？](#)

这问题其实是在问提示词工程的系统化设计思路，不只是随便写句话。

1) 首先看维度。一个有效的提示词必须明确任务目标、输入约束和输出格式。任务类型决定结构，比如分类、生成、抽取需要的模板完全不同。上下文信息也得给足，模型才能理解语义边界。还要考虑安全合规要求，像敏感词过滤、价值观对齐这些都得前置控制。

2) 再说模板字段。我们常用的提示词模板包含这几个关键部分：角色定义，说明这个 prompt 是给谁用的；背景描述，交代清楚业务场景；指令主体，这是核心，告诉模型具体要做什么；示例样本，few-shot 的例子能显著提升准确率；输出约束，比如 JSON 格式、字数限制、枚举值范围；最后是异常处理机制，定义模糊输入时的降级策略。

代码示例长这样：

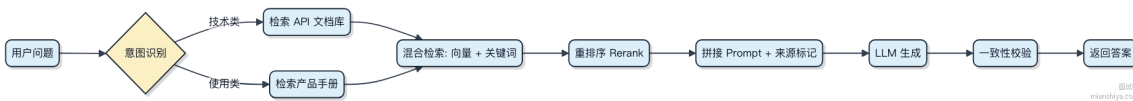
```
prompt = """
角色：电商客服助手
背景：用户在商品详情页发起咨询
指令：根据商品信息和用户问题，生成专业且友好的回复
输入：
  - 商品标题：{title}
  - 用户问题：{query}
输出格式：JSON，包含 reply 字段，不超过 150 字
"""
```

这种结构能让大模型稳定输出符合预期的结果，而不是靠运气。

你有多个知识库，做 RAG 的时候，怎么保证查询效率和准确性兼容，并尽可能减少幻觉？

多个知识库做 RAG，关键是在检索阶段就控制好数据的“来源质量”和“语义相关性”，否则后面生成再怎么优化也压根不经过。

- 1) 先做知识库的预分类或打标。比如用 Milvus 或 Elasticsearch 存向量时，给每个文档加 metadata 标记来源类型（如产品文档、客服记录、API 手册）。查询时先根据 query 的意图路由到最可能相关的库，避免全量检索拖慢速度。
- 2) 用 **混合检索** 策略。单纯靠向量可能召回语义相近但事实错误的内容，容易引发幻觉。结合关键词匹配（BM25）和向量相似度，像 Weaviate 或 Vespa 都支持这种 hybrid search，能同时抓住字面匹配和深层语义，准确率提升明显。
- 3) 检索后做重排序（rerank）。别拿到 top-k 就直接喂给 LLM，用一个轻量级模型比如 BGE-reranker 对结果按相关性重新打分，把真正靠谱的段落往前排。这一步能过滤掉不少“看起来像对其实不对”的内容。
- 4) 在 prompt 中显式标注信息来源。把检索到的文本块加上出处标签，比如 [来自：API 手册 v3.2]，让大模型知道依据从哪来，生成时更倾向引用原文，减少脑补。
- 5) 最后加一层验证机制。对关键回答，可以用小模型做一致性校验，比如把生成结果和原始文档片段做一次 entailment 判断，不一致就触发人工审核或 fallback 回答。



Agent 死循环问题有遇到过吗？如何解决？

Agent 死循环通常发生在字节码增强逻辑里，比如用 Java Agent 做监控或 APM 时，不小心把自己 instrument 的类又触发了新的 transform，结果无限加载 class，CPU 直接打满。

最常见的场景是，你在 `ClassFileTransformer` 里处理某个类的时候，执行了某些代码，这些代码又动态生成类或者触发类加载，而你的 transformer 没做过滤，就会重新进入 transform 方法，形成递归调用。特别是用到 ASM、CGLIB、Javassist 这些框架时很容易中招。

解决办法其实就两条：

- 1) 加 **类名过滤**，直接排除不必要的类，尤其是你 agent 自身的包路径，比如 `com.mycompany.agent.**` 全部 return null 不做处理。
- 2) 用 **本地线程标记**（ThreadLocal）标记当前是否已经在 transform 流程中，如果是，直接跳过。这个特别关键，因为很多间接类加载是你控制不了的。

```
static final ThreadLocal<Boolean> transforming = new ThreadLocal<>();

public byte[] transform(ClassLoader loader, String className, Class<?>
    classBeingRedefined,
    ProtectionDomain protectionDomain, byte[] classfileBuffer) {
    if (transforming.get() != null) return null;
    try {
        transforming.set(true);
        // 执行实际的字节码修改逻辑
        return doTransform(loader, className, classfileBuffer);
    } finally {
        transforming.remove();
    }
}
```

另外，像 SkyWalking、Pinpoint 这些 APM 工具都遇到过这类问题，它们的源码里都有对应的 guard 机制防止重入。参考它们的实现能少踩很多坑。



如果一个GPU集群的LLM处理能力为1000tokens/s，那1000个用户同时并发访问，响应给每个用户的性能只有1 token/s吗？怎么分析性能瓶颈

不一定每个用户就只能分到 1 token/s，实际性能取决于调度策略、批处理能力和请求的负载特征。

大模型服务不像传统 Web 服务器那样“一个请求占一个线程”，它走的是 **动态批处理（Dynamic Batching）** 路子。比如用 vLLM、TensorRT-LLM 这类推理框架时，多个用户的请求会被合并成一个 batch 一起跑，显存和计算单元利用率更高。

关键点在于：

1) 如果这 1000 个用户都是刚发请求，还没开始生成内容（prefill 阶段为主），那系统可以一次性把他们的 prompt 做 batch 处理，吞吐可能接近峰值 1000 tokens/s，平均下来每人几十甚至上百 token/s 都有可能。

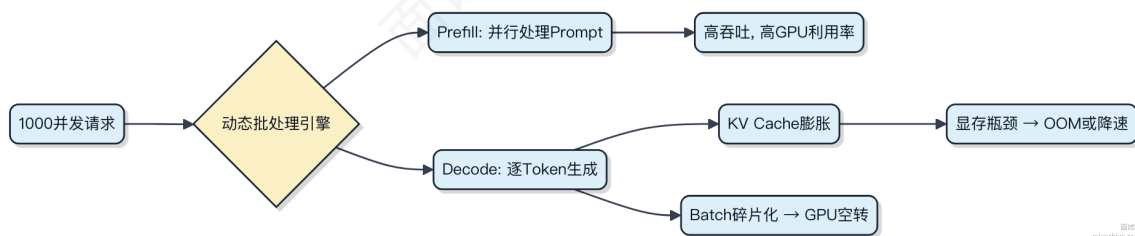
2) 但到了 decode 阶段，每个请求要逐 token 生成，且长度不一，这时候 batch size 会慢慢被“拖慢”。长文本、高采样参数（如 temperature 高）的请求会拉低整体速度。

3) 真正的瓶颈往往不是算力，而是显存带宽和 KV Cache 管理。每个并发请求都要保留 KV Cache，1000 个并发意味着巨大的显存压力，搞不定就会 OOM 或被迫丢请求。

举个例子，A100 显存 80GB，跑 Llama3-8B，每个请求平均占 2GB KV Cache，理论上最多撑 40 个并发。超过就得靠 PagedAttention（像 vLLM 实现的）来分页管理，否则压根不经过计算就挂了。

所以性能瓶颈通常出现在：

- KV Cache 显存占用过大
- 小 batch 或不规则请求导致 GPU 利用率上不去
- 请求排队时间远超生成时间



优化方向是：用 PagedAttention 降低显存压力，启用连续批处理（Continuous Batching），控制最大生成长度，避免小请求拖垮整体延迟。

请举例说明假设在电商系统中，哪些功能适合直接使用大模型完成，哪些需要结合工程化手段？

电商系统里大模型能直接干的活，其实集中在非结构化数据处理和生成类任务上。比如客服场景，用户问“这件衣服能机洗吗”，传统方案得靠关键词匹配或意图识别走规则，现在可以直接把对话历史、商品详情喂给大模型，让它生成自然语言回复。类似地，商品标题生成、详情页文案润色、营销短信个性化撰写，这些内容创作类需求，大模型开箱即用效果就不错。

但涉及确定性逻辑和系统协同的功能，光靠大模型压根不经过关。比如下单扣库存，你不可能让大模型去决定减不减 1，这种操作必须由事务型数据库保证原子性。再比如推荐系统，虽然可以用大模型做用户兴趣向量化，但最终的千人千面排序还得结合 Flink 实时特征、HBase 用户画像、以及 Redis 缓存的热点商品列表，靠一串工程链路拼起来。

还有个典型例子是搜索相关性优化。你可以用大模型理解用户 query 的语义，比如把“苹果”识别为水果还是手机，这部分可以单独跑一个推理服务。但整个搜索链路依然要走 Elasticsearch 倒排索引召回，再用模型打分重排，最后通过熔断降级、缓存预热这些手段保障 SLA。

简单说，大模型适合处理模糊边界的问题，而订单、支付、库存这些核心链路，还得靠传统工程手段扛住。

假设请你设计一个医疗问诊系统，如何平衡AI幻觉带来的风险与效率提升？需要哪些技术手段？

医疗问诊系统里，AI 效率是真高，一个模型能同时看几十路问诊，但幻觉问题可不敢马虎。说错症状、开错建议，那是要出事的。

关键在于不能让大模型直接输出诊断结论。得给它套上几层“安全锁”。

- 1) 所有用户输入先过一遍意图识别和实体抽取，用微调过的小模型把“我头疼三天了”转成结构化数据，比如 {symptom: headache, duration: 3 days}。这步很稳，不靠生成。
- 2) 结构化数据进知识图谱推理。咱们有医学本体库，像 UMLS 或自建的疾病-症状-检查关系网。模型查“头痛+发烧”可能关联流感、脑膜炎，但不会瞎编个“火星病毒”。
- 3) 生成回复时，强制走检索增强（RAG）。答案必须来自权威指南，比如 UpToDate 或卫健委诊疗规范，向量库里只放审核过的文档。大模型只负责把检索到的内容用口语化组织出来，不准自由发挥。
- 4) 最后加一层规则引擎做合规校验。比如涉及处方药，必须提示“请线下就诊”，涉及急症关键词自动转人工。



说白了，大模型只干“语言润色”的活，诊断逻辑全由确定性系统驱动。效率归AI，安全归规则，各司其职。

设计智能客服系统时，如何通过知识库构建解决长尾问题？请描述具体实现步骤

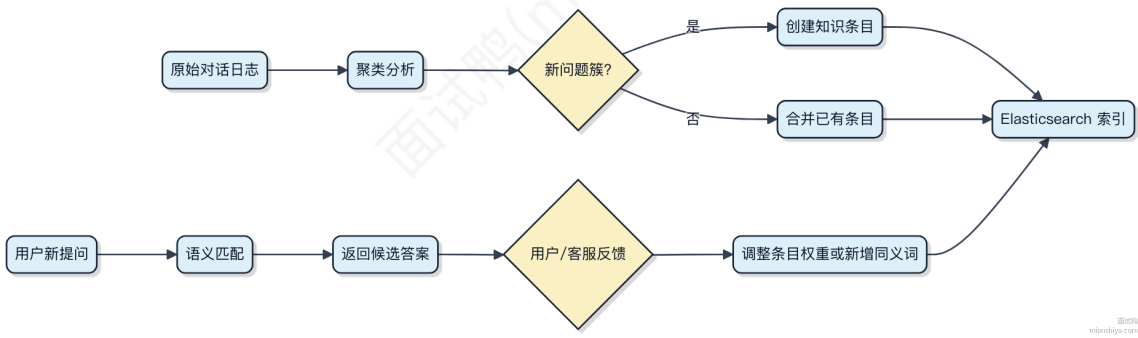
智能客服的长尾问题指的是那些发生频率低、但种类繁多的用户提问，比如“怎么退订海外会员的自动续费”。这类问题单个看很少见，加起来能占到 30%-40% 的咨询量，靠人工维护规则搞不定。

解决思路是把散落的问题沉淀成结构化知识库，让系统能自动匹配相似问法。第一步，收集历史工单、客服聊天记录，用 NLP 做聚类分析，把语义相近的问题归到一起，比如“取消订阅”、“退会员”、“停自动扣费”都归为同一类。

第二步，定义知识条目结构，每个条目包含标准问法、同义问法扩展、答案内容、适用业务场景标签。可以用 Elasticsearch 建索引，用户提问进来后，先做分词和语义向量化，再在知识库中找最相似的标准问法。

第三步，引入反馈闭环。线上每次匹配结果被客服采纳或用户点击“有帮助”，就记一次正向反馈；如果没命中或被修正，就触发人工复核机制，定期更新知识条目。

- 1) 聚类原始问题发现潜在长尾
- 2) 构建带同义词扩展的知识条目
- 3) 语义检索 + 向量相似度匹配
- 4) 通过用户行为持续优化覆盖



关键是别指望一步全覆盖，而是靠这个闭环让知识库自己“长大”。

什么是低秩适配（LoRA）技术？如何结合 LoRA 技术进行微调？

低秩适配（LoRA）的核心思路是，大模型微调时参数量太大，训练慢、显存吃紧，干脆不直接改原始权重，而是用低秩矩阵去近似增量。

模型原本的权重矩阵 W 是个大胖子，比如 768×768 。LoRA 不动它，转而引入两个小矩阵 A 和 B ，维度分别是 $768 \times r$ 和 $r \times 768$ ，其中 r 很小，一般取 8 到 64。训练时只更新这两个小矩阵，前向传播时等效注入 $\Delta W = A \times B$ 。

这样显存和计算量就从和原矩阵成正比，变成和 r 相关。比如 $r=8$ ，参数量直接降到原来的 $1/96$ ，训练速度翻倍都不止。

结合 LoRA 做微调，流程也简单：

- 1) 冻结原始模型所有权重
- 2) 在指定层（比如注意力中的 Q、V 投影）插入 LoRA 模块
- 3) 只训练新增的小矩阵，其余参数不动
- 4) 推理时把 $A \times B$ 加到原权重上，或者直接合并进原权重

主流框架都有支持。Hugging Face 的 `peft` 库几行代码就能上：

```
from peft import LoraConfig, get_peft_model

lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.1,
    bias="none",
    task_type="CAUSAL_LM"
)
model = get_peft_model(model, lora_config)
```

适合资源有限、多任务切换的场景。像微调一个 7B 模型，用 LoRA 能在单卡 24G 显存下搞定，否则根本搞不定。

微调的过拟合风险如何通过正则化缓解？

微调时模型已经见过大量训练数据，参数调整幅度小但密集，容易记住训练集的特定模式，导致在新数据上表现下滑。正则化的核心作用就是在损失函数或训练过程中加入约束，让模型别学得太“死”。

- 1) L2 正则（权重衰减）是最常用的手段。它在损失函数里加上所有权重的平方和，迫使参数值趋向更小。比如 Hugging Face 的 `Trainer` 里设置 `weight_decay=0.01`，就能防止某些权重变得过大，提升泛化能力。

```
training_args = TrainingArguments(
    output_dir="output",
    weight_decay=0.01
)
```

- 2) Dropout 在微调阶段依然有效。虽然预训练模型内部已经固化了结构，但在分类头甚至最后几层中间加 Dropout，能让输出不依赖于少数神经元。一般保留率设在 0.1~0.3 就够了。

- 3) 早停（Early Stopping）特别适合微调场景。因为微调轮次少，可能只跑 3~5 个 epoch，验证集性能一旦下降就终止，能避免多走一两步掉进过拟合坑里。

4) 还有像 **标签平滑** (Label Smoothing) 这种软化监督信号的方法。它不让模型对真实标签过于自信，间接抑制过拟合。在分类任务中设 `label_smoothing_factor=0.1` 是常见做法。

这些方法本质都是给学习过程加点“不确定性”或“惩罚”，让模型别把微调当背书，稍微收着点劲。

请详细讨论微调时如何防止灾难性遗忘问题？

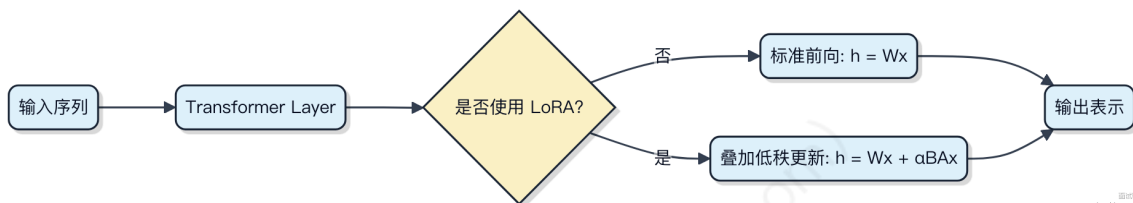
微调大模型时，灾难性遗忘的本质是模型参数被新任务数据剧烈调整，导致旧知识的表征能力崩塌。关键不是“记住所有旧数据”，而是让模型在适应新任务的同时，保留对原始分布的泛化能力。

1) 降低学习率是最直接的手段。预训练阶段用了海量数据，参数已经收敛到一个较优的全局位置。微调时如果用大学习率，梯度更新会把参数推出这个区域。一般建议微调学习率设为预训练的 1/10 到 1/100，比如从 $3e-4$ 降到 $3e-5$ 或 $1e-5$ 。

2) 使用 **Adapter** 模块是一种参数隔离策略。只在 Transformer 层间插入小型神经网络，冻结原始主干参数。训练时梯度只流经这些小模块，主干权重压根不更新。这样既适配了新任务，又完全保留了原模型的知识结构。HuggingFace 的 PEFT 库就支持这类方法。

3) **LoRA** (低秩适应) 更轻量。它不加新层，而是给注意力权重矩阵注入低秩分解矩阵。假设原始权重是 W ，训练的是 $\Delta W = BA$ ($A \in \mathbb{R}^{[r \times d]}$, $B \in \mathbb{R}^{[d \times r]}$)， r 远小于 d (比如 $r=8$, $d=4096$)。更新时 $W' = W + \alpha * BA$ ，主干参数不变，只训练 A 和 B 。这种方法显存占用少，推理还能合并权重，适合大规模部署。

4) 数据层面可以混合一定比例的通用语料。比如每 10 个微调样本中加入 1 个来自原始预训练分布的句子，让梯度方向不至于完全偏离。不过要小心引入噪声影响目标任务效果。



这些方法的核心思想都是**限制参数搜索空间**，不让优化过程肆意修改已学好的特征提取器。

在多模态微调（如图文生成）中，如何确保文本和图像数据的对齐质量？

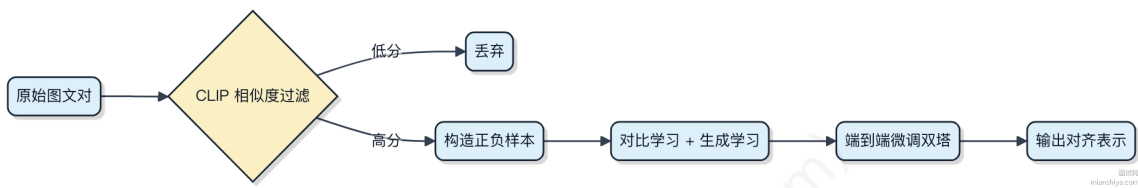
多模态微调里，文本和图像的对齐质量直接决定模型能不能准确理解“图在说什么，话在描述谁”。如果数据没对齐，模型学到的就是错配关系，比如把“猫坐在沙发上”跟一张狗的照片强行关联，后续生成或推理全崩。

1) 首先靠**高质量数据清洗**。原始图文对往往来自网页，alt-text 和图片内容不一致太常见。可以用预训练的 CLIP 模型做初筛，计算图文相似度，低于阈值（比如 0.2）的直接过滤。CLIP 本身是跨模态编码器，能给出语义匹配分数，这一步能干掉明显噪声。

2) 然后是**增强对齐信号**。单纯用原始图文对微调，监督信号弱。可以在输入侧构造负样本，比如同一个 batch 里拿其他样本的文本当负例，做对比学习 (ITC loss)，逼模型分辨“哪个文本真配这张图”。BLIP、ALBEF 这些模型就这么干的。

3) 微调策略上，**不要只微调分类头**，要端到端微调图像编码器和文本编码器。尤其是 ViT 的高层特征和文本 token 要能 cross-attention 对上。可以用 attention 可视化检查：生成文本时，模型是不是真的关注图中对应区域。

4) 最后加点**生成式损失**，比如在 decoder 阶段用语言模型目标重建描述，让模型不仅判别还得会说。这样双重约束，对齐更稳。

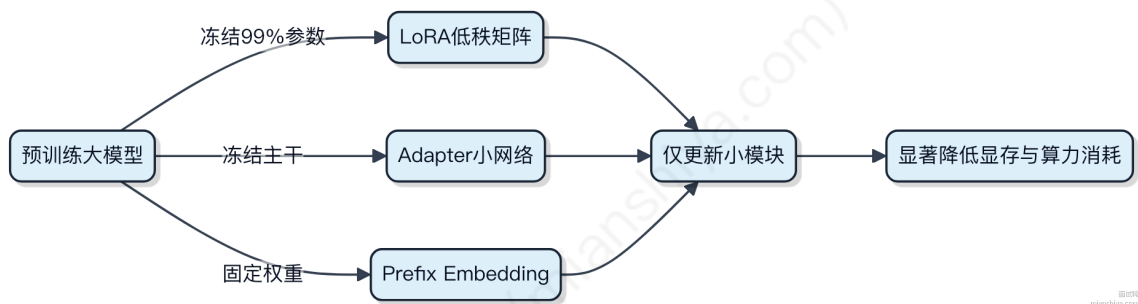


参数高效微调（PEFT）如何减少计算成本？

参数高效微调的核心思路是，不碰预训练模型的大部分权重，只动其中一小撮关键参数。这样一来，显存占用和计算量都大幅下降。

- 1) 冻结主干：把大模型的绝大多数参数锁死，比如 LLaMA 的 70 亿参数里，可能只放开不到 1% 去更新。剩下的全当静态特征提取器用。
- 2) 引入可训练模块：像 **LoRA** 这种方法，会在注意力层的权重旁加个低秩矩阵做增量。训练时只优化这个小矩阵，原始权重压根不参与梯度更新。假设原矩阵是 $\mathbb{R}^{5120 \times 5120}$ ，LoRA 只学两个 $\mathbb{R}^{5120 \times 8}$ 的小矩阵，参数量从千万级干到几万。
- 3) 适配器注入：在 Transformer 层之间塞个小的前馈网络模块，叫 Adapter。它通常就一两层，宽度也小，训练时只调这部分，其他照旧。像 BERT 这类模型用 Adapter 微调，能省下 70% 以上的训练资源。
- 4) 前缀微调：给输入前面挂一串可学习的虚拟 token，叫 prefix。训练时只更新这些 token 的 embedding，模型主体完全不动。T5、GPT 这类自回归模型常用这招。

这些方法都能做到单卡微调大模型。比如用 LoRA 调 LLaMA-2-13B，在 A100 上也能跑起来，显存直接从几十 GB 掉到 10G 内。



冻结层在微调中的作用是什么？

冻结层在微调时，其实就是把预训练模型里的一部分网络参数固定住，不让它们更新。你只让最后几层或者指定的层去学习新任务的数据，这样能省下大量计算资源。

- 1) 最常见的情况是用 BERT、ResNet 这类大模型做下游任务。比如你拿 ImageNet 上训好的 ResNet50 去做猫狗分类，前面的卷积层已经学到了边缘、纹理这些通用特征，压根不需要重学。把这些层冻住，只微调最后的全连接层，几十个 epoch 就能收敛，显存和时间都扛得住。
- 2) 另一个关键是防止过拟合。你的小数据集可能就几千张图，如果放开所有层去更新，模型很容易记住噪声，泛化能力直接崩掉。冻结大部分层相当于限制模型容量，逼它只调整高层语义适配新任务。

3) 也有分阶段微调的做法：先冻结主干训 head，再解冻部分层用更小学习率细调。像 Hugging Face 的 Transformers 库里，`.requires_grad_()` 控制起来也就一行代码的事。

```
# 冻结 BERT base 层
model = BertForSequenceClassification.from_pretrained('bert-base-uncased')
for param in model.bert.parameters():
    param.requires_grad = False # 梯度不计算，节省内存
```

这种策略在 NLP 和 CV 里都成了标配，尤其是数据少、又想用大模型的时候，基本靠它稳住局面。

为什么需要混合精度训练？

混合精度训练解决的是深度学习训练中的计算效率与显存占用问题。现代GPU如A100、V100都配备了Tensor Core，专门为半精度浮点数（FP16）运算优化，单次可处理更多数据，算得更快。

1) FP16相比FP32占用带宽和显存少一半，同样的batch size能跑更小的显存，或者在显存不变时增大batch size提升训练稳定性。

2) 但全用FP16会出问题，比如梯度下溢、权重更新精度不够导致模型不收敛。所以得保留一部分关键计算用FP32，比如梯度累加、权重更新、loss缩放。

3) 实际做法是：前向和反向传播用FP16加速，参数副本保持一份FP32的主拷贝用于更新，这就是**AMP (Automatic Mixed Precision)** 的核心机制。PyTorch里一行 `scaler = torch.cuda.amp.GradScaler()` 就能开启。

像BERT、ResNet这类大模型训练，基本都默认开启混合精度。NVIDIA的Apex库和原生AMP都支持得很好。不用的话，等于白白浪费30%以上的训练速度。



模型输出重复和幻觉如何微调解决？

模型输出重复和幻觉，本质是训练数据、生成策略和微调方式共同作用的结果。想靠微调彻底解决不现实，但能显著缓解。

1) 重复问题，常见于解码阶段贪心搜索。beam search 如果宽度太小，容易陷入循环。可以在推理时引入 **n-gram 重复惩罚**，比如 Hugging Face 的 transformers 库里设置 `no_repeat_ngram_size=2`，让模型避开已生成的词组。微调时也可以在 loss 上做文章，对重复 token 加权惩罚。

2) 幻觉更棘手，模型把参数里的统计规律当事实输出。指令微调 (Instruction Tuning) 是个办法，用高质量问答对训练，比如 Alpaca 数据集格式，教会模型“不知道就说不知道”。关键是数据质量，10 万条精准样本比 100 万条噪声数据管用得多。

3) RLHF 也能压幻觉，InstructGPT 就这么干的。用人类偏好打标数据训练奖励模型，再用 PPO 调整生成策略，让输出更可信。但这套流程成本高，一般公司搞不定。

代码上，Hugging Face 的 Trainer 配合 `DataCollatorForLanguageModeling` 做微调很顺手。推理时控制 `top_p` (0.9) 和 `temperature` (0.7)，别让模型太放飞。

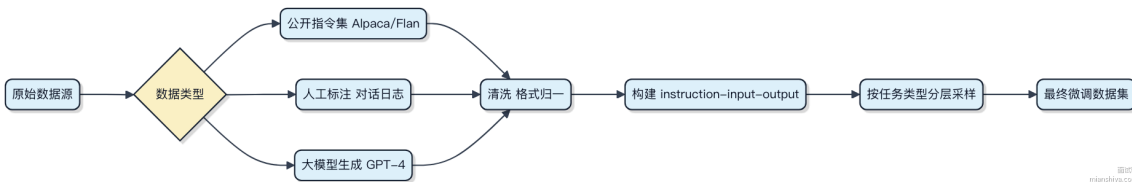


说到底，微调能改善，但没法根治。工程上得配合检索增强（RAG），让模型查实时数据，而不是全靠记忆。

SFT 指令微调数据如何构建？

SFT（Supervised Fine-Tuning）指令微调数据的构建，本质是把模型要学的任务表达成“指令-输入-输出”的三元组结构。关键不是堆数据量，而是让样本覆盖多样任务类型和表达方式。

- 1) 一条标准样本包含三个部分：**instruction** 是具体任务描述，比如“翻译成英文”；**input** 是待处理内容，可以为空；**output** 是期望的模型输出。例如 instruction="摘要生成", input="一段新闻文本", output="生成的简短摘要"。
- 2) 数据来源主要有几种路径。可以用公开指令数据集如 Alpaca、Flan-v2 做基础，再结合业务场景构造垂直领域样本。比如客服场景，收集真实用户问题和人工回复对，包装成指令格式。也可以用大模型自动生成指令样本，通过 prompt 让 GPT-4 产出“问题+答案+上下文”，控制多样性和质量。
- 3) 质量比数量重要。避免重复、歧义或错误标注。建议做分层设计：通用能力、领域知识、业务特有逻辑各占一定比例。一般几千到几万条高质量样本就能起效，上百万条反而可能引入噪声。
- 4) 最后要做格式归一和去重。统一 prompt 模板风格，过滤低信息密度样本，比如全是停用词或模板化回复的数据。整个过程其实是为模型“造考场”，题目得能考出真本事。



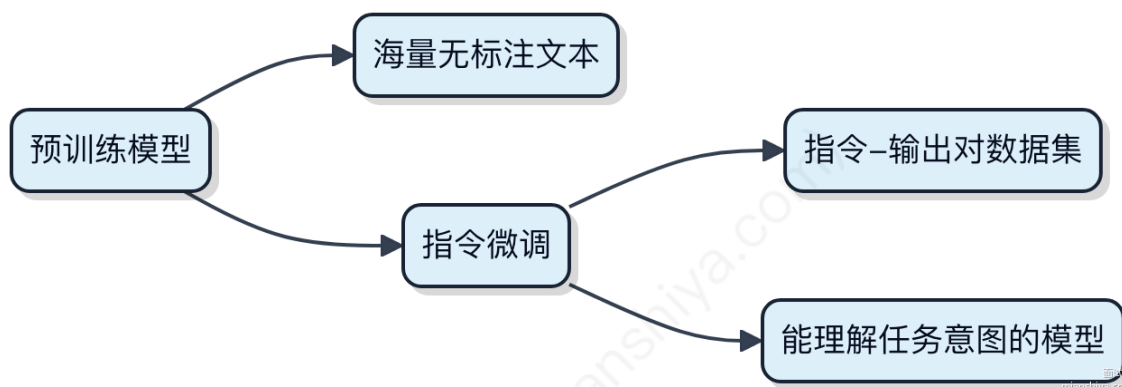
指令微调的好处？

指令微调其实是让预训练模型更会“听人话”的关键一步。模型在大规模数据上预训练完，虽然学到了丰富的语言知识，但并不清楚用户到底想让它干什么。这时候通过指令微调，用成对的“指令-期望输出”数据去微调，它就能学会理解任务意图。

- 1) 模型能准确理解任务形式。比如给一条指令“把下面这句话翻译成英文”，模型知道接下来要处理的是翻译，而不是摘要或改写。这种泛化能力在多任务场景下特别有用。
- 2) 减少对示例的依赖。没有指令微调的模型，往往需要上下文里给出几个例子（few-shot）才能干活，而微调后的模型可以直接响应指令，输入更简洁。
- 3) 行为更可控。你可以通过微调引导模型输出特定格式，比如 JSON、有序列表，或者避免生成某些敏感内容。像 Alpaca、Dolly 这些开源模型，都是基于 LLaMA 做了指令微调才变得可用。

举个简单例子，微调前你问“写个快排”，它可能只解释原理；微调后它大概率直接给你一段可运行的代码。

不过也别指望它能完全替代工程逻辑。指令微调解决的是“意图对齐”，不是“能力增强”。模型本身不具备的能力，比如数学推理、代码执行，光靠指令微调是搞不定的。



如何让 LLM Agent 具备长期记忆能力？

让 LLM Agent 有长期记忆，关键不是靠模型本身记住东西，大模型的上下文窗口再大，也就几十k到百k token，根本存不下长期数据。真正的解法是**外挂存储 + 检索机制**。

1) 把 Agent 历史交互、关键决策、用户偏好这些信息存到外部数据库。可以用向量数据库比如 **Chroma**、**Pinecone** 存语义向量，方便后续语义检索。同时用传统数据库如 **PostgreSQL** 存结构化元数据，比如时间、对话ID、标签等。

2) 每次新请求进来，先根据当前输入做语义匹配，在向量库中查最相关的几段历史记录。这个过程叫 **Retrieval-Augmented Generation (RAG)**，把查到的内容拼进 prompt，一起喂给 LLM。

3) 检索策略要设计好。不能每次都拉全部历史，会炸上下文。可以按时间衰减加权，或者用摘要压缩老记忆，比如每天生成一个当日行为摘要存进去。

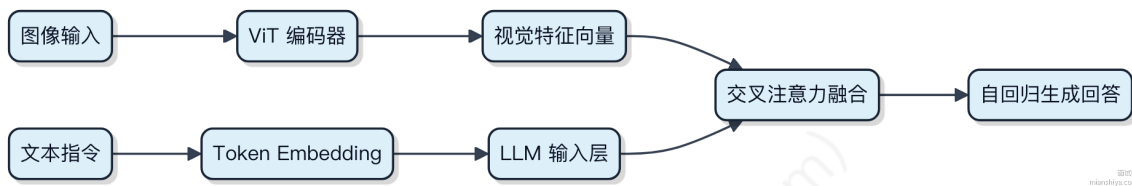
```
// 伪代码示意：记忆检索核心逻辑
List<Memory> recent = memoryStore.findByTimeRange(last24Hours);
List<Memory> relevant = vectorDB.similaritySearch(currentInput, topK=3);
Prompt prompt = buildPromptWithMemories(currentInput, recent, relevant);
String response = llm.generate(prompt);
memoryStore.save(new Memory(timestamp, currentInput, response)); // 写回
```

这么一来，Agent 就能“记得”几个月前的事，还能理解“上次你说不喜欢红色”这种指代。本质上，长期记忆是个读多写少的系统，重点在快速定位有用信息，而不是全量回放。

LLM Agent 在多模态任务中如何执行推理？

多模态任务里，LLM Agent 的推理不是简单看图说话，而是把文本、图像、音频这些不同模态的信息统一拉到一个向量空间里做对齐。模型在预训练阶段就见过大量图文对，比如用 CLIP 这类结构把图片和文字映射到同一语义空间，这样一张“狗追球”的图，能和对应描述在特征上挨得特别近。

1) 输入进来时，图像走视觉编码器（比如 ViT）转成 patch embeddings，文本走 tokenizer 转成 token embeddings，两者长度不一样没关系，关键是通过交叉注意力让 LLM 主干能“看见”图像特征。2) 推理过程中，LLM 不是只靠记忆生成答案，而是结合上下文做因果推断。比如问“图中动物在干什么”，它会关联“奔跑”、“草地”、“球体运动轨迹”这些视觉线索，生成“狗在追球”而不是“狗在睡觉”。3) 有些复杂任务还会引入外部工具调用，比如先识别图像中的数学公式，再丢给 SymPy 计算，最后把结果整合进回答——这一步就是 agent 的规划能力体现。



这类系统典型代表是 LLaVA、Qwen-VL，它们在架构上做了端到端训练，确保语言模型能真正理解而不是“猜”图像内容。纯 prompt 工程压根搞不定跨模态语义鸿沟，必须依赖对齐训练。

LLamaIndex 如何与 LangChain 结合？

LLamaIndex 和 LangChain 不是二选一的关系，更多时候是配合使用，一个搞数据连接，一个做流程编排。

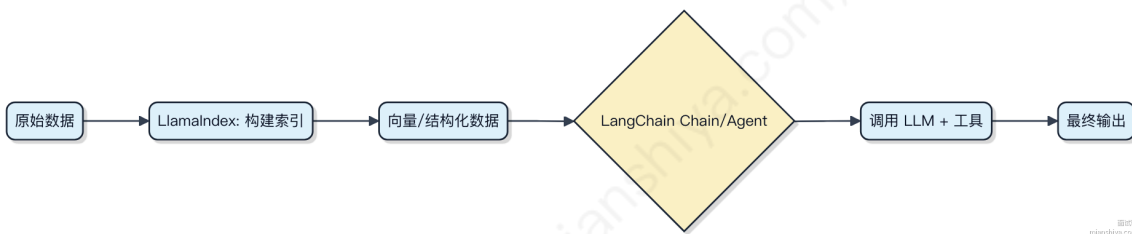
1) LlamaIndex 的核心是**索引**。它擅长把非结构化数据比如 PDF、数据库、网页内容，变成向量或结构化索引，方便大模型高效检索。像你用它接 Notion、Slack 数据，做 RAG（检索增强生成），效果很直接。

2) LangChain 重点在**链式流程**。你要串起提示词、工具调用、记忆管理、多个 LLM 步骤，LangChain 提供了统一的抽象和调度能力。比如你做一个客服机器人，要先查订单，再生成回复，还得记住上下文，这就得靠 Chain 或 Agent 模式。

它们怎么结合？常见两种方式：

第一种，用 LlamaIndex 做数据源，喂给 LangChain 流程。比如你在 LangChain 里定义一个 RetrievalQA 链，retriever 直接对接 LlamaIndex 构建的 VectorStoreIndex。

第二种，反过来，在 LlamaIndex 的 Query Engine 里嵌入 LangChain 的组件。比如你让 LlamaIndex 查完数据后，用 LangChain 的提示模板重写查询，或者调用外部 API 补充信息。



简单说，LlamaIndex 解决“**从哪找数据**”，LangChain 解决“**怎么一步步处理**”。一个负责精准拉取，一个负责逻辑流转，搭在一起能快速做出复杂应用。

AutoGPT 如何实现自主决策？

AutoGPT 本身并不是一个标准化的开源框架，而是一类基于大语言模型（LLM）实现自主任务分解与执行的实验性系统。它的“自主决策”能力，其实是通过**提示工程 + 外部工具调用 + 反馈循环**构建出来的行为模式。

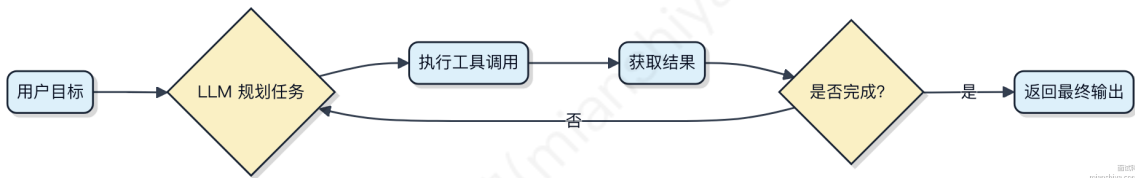
1) 它的核心机制是把目标输入给 LLM，让模型自己拆解成可执行的子任务。比如你要它“开发一个待办事项应用”，它会先决定需要创建项目目录、写前端代码、设计API等。

2) 每完成一步，系统会把结果反馈回去，LLM 再判断下一步做什么，或者是否需要修正。这个过程就像不断自问自答：我做了什么？现在处于什么状态？接下来该干什么？

3) 决策落地依赖外部工具支持。LLM 本身不会写文件、调接口，但它可以输出结构化指令，由代理程序去执行 shell 命令、读写代码文件、调用搜索引擎或 API。像 LangChain 这类框架就提供了这类动作的封装。

4) 自主性是有边界的。它没法真正理解因果，只是根据上下文概率生成“合理”的下一步。一旦任务链偏离预期，容易陷入无限循环或重复操作。实际使用中经常需要人工干预止损。

整个流程本质上是一个**带记忆的 ReAct 模式** (Reasoning + Acting)，靠 prompt 把推理和动作绑定在一起。典型结构如下：



所以别被“自主”俩字唬住，它压根不经过真实逻辑判断，全靠模型“觉得下一步应该这么做”。复杂任务搞不定，长期运行稳不住，但做点小自动化脚本生成还能扛住。

本资源来自面试鸭：<https://www.mianshiya.com>

推荐更多免费学编程资源：

1. 编程导航学习网站：[学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库：[开始刷题，面试遇原题!](#)
3. 精选简历模板大全：[1 分钟搞定简历!](#)
4. AI 资源导航网站：[获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试：[随时随地提升面试能力](#)